Lecture 1

Introduction to Deep Learning for Language Modeling

MAI554: Deep Learning for NLP

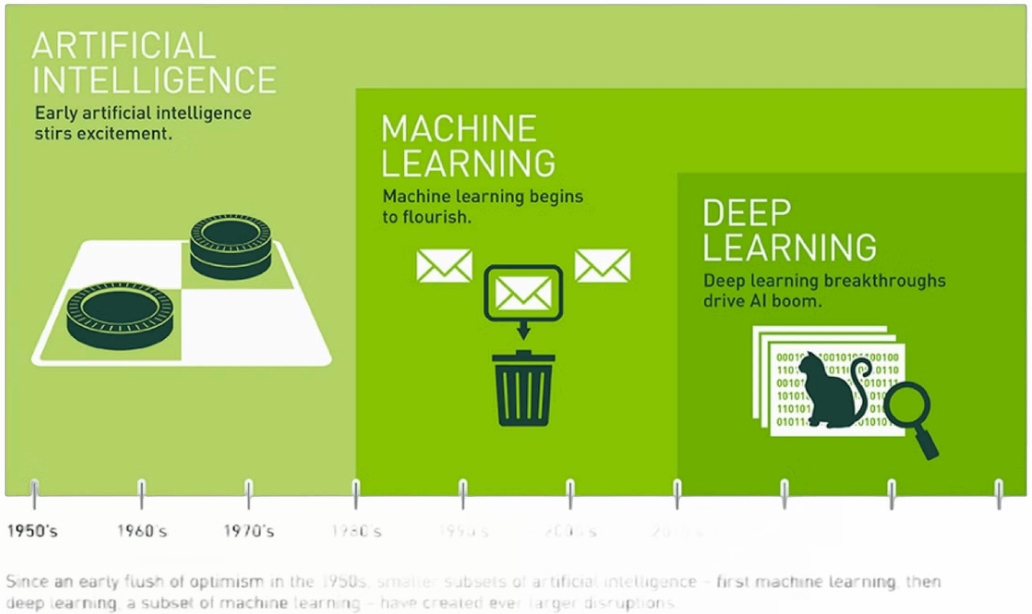
Prof. Anis Koubaa

Professor

College of Engineering
Alfaisal University

akoubaa@alfaisal.edu

From AI to Deep Learning



Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

1

Artificial Intelligence

The broad concept of machines being able to carry out tasks in a way that we would consider "smart"

2

Machine Learning

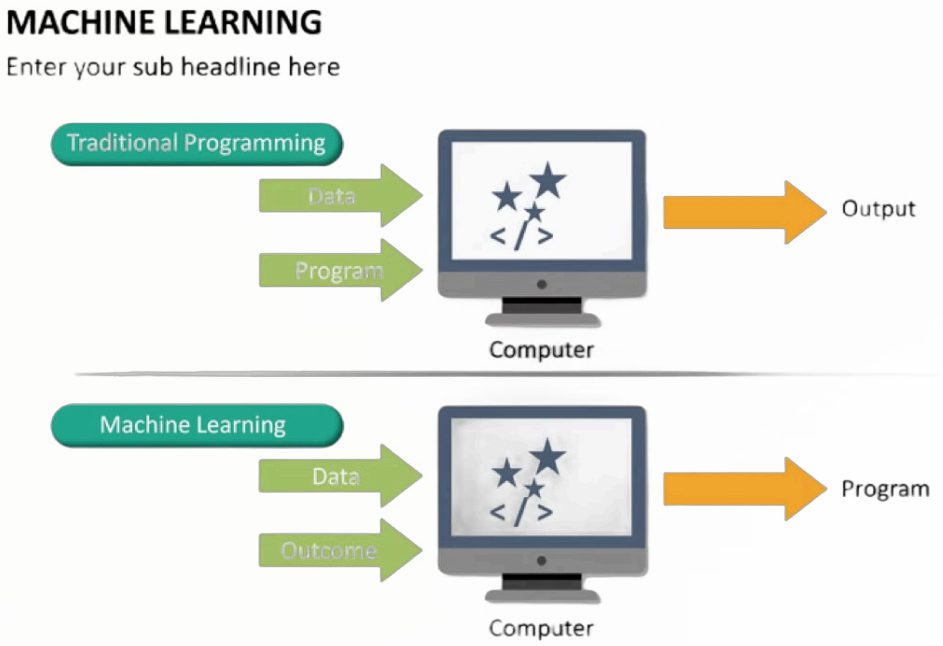
Systems that can learn from data without being explicitly programmed

3

Deep Learning

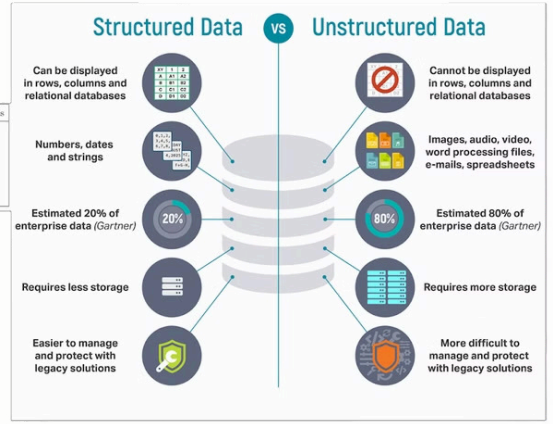
Advanced ML using neural networks with multiple layers to process complex patterns

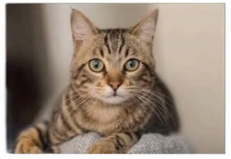
Traditional Learning vs. Machine Learning

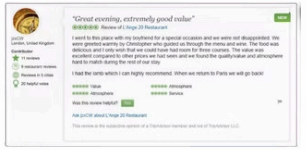


- **Rule-based** vs **Pattern-based:** Traditional uses explicit instructions, ML discovers from data
- **Improvement Method:** Traditional requires manual updates, ML adapts automatically
- **Development Process:** Traditional needs step-by-step coding, ML builds own algorithms
- **Pattern Recognition:** Traditional handles simple rules, ML excels with complex relationships

Data	Number of timepoints	Number of nodes
Antibiotics	56	386
Preterm births	216	318
Housing prices	254	944
Bikesharing	24	819
Global patterns	500	51







Data Categories

Structured Data

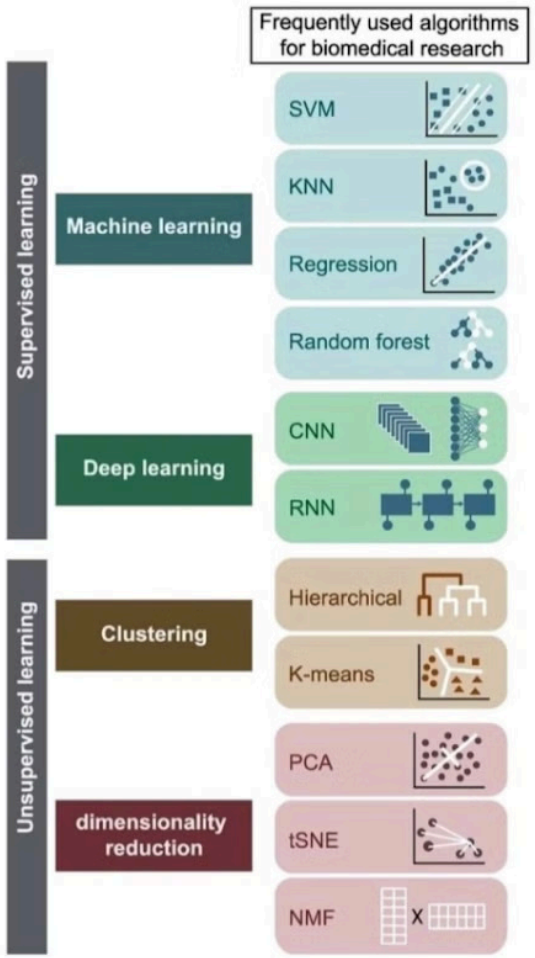
- Organized in **predefined formats**
- Stored in **databases** and **spreadsheets**
- Easy to **search** and **analyze**
- Examples: **Excel files**, **SQL databases**

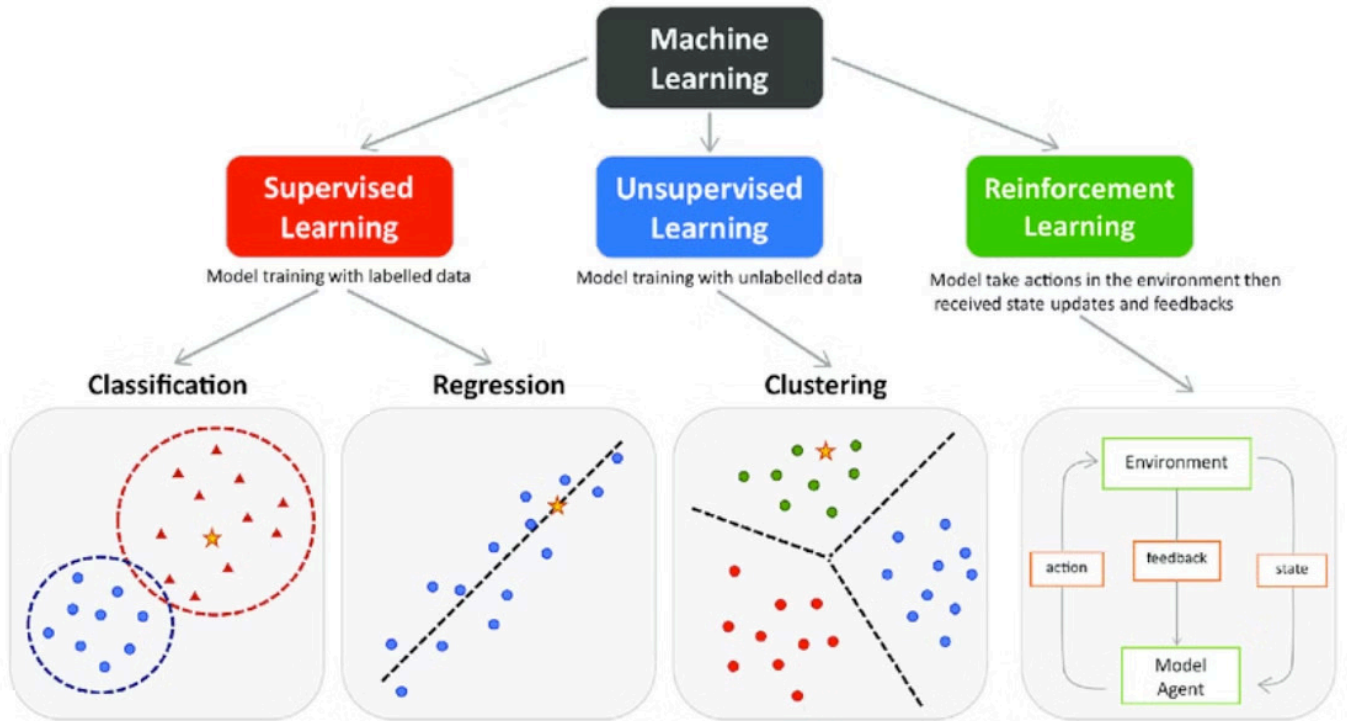
Unstructured Data

- No **predefined organization**
- Includes **text**, **images**, **videos**
- More complex to **analyze**
- Examples: **Emails**, **social media posts**

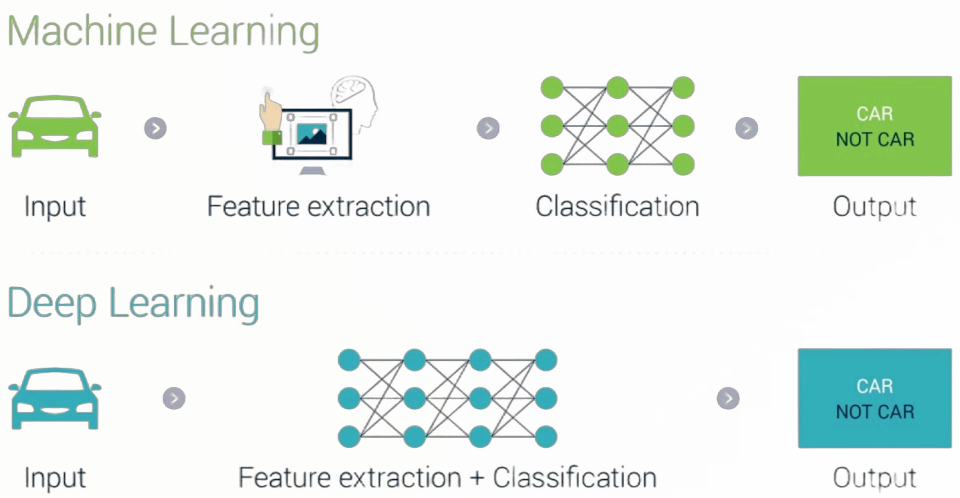
Understanding these **data types** is crucial for choosing appropriate **AI** and **ML** approaches.

MACHINE LEARNING
SUPERVISED vs. UNSUPERVISED



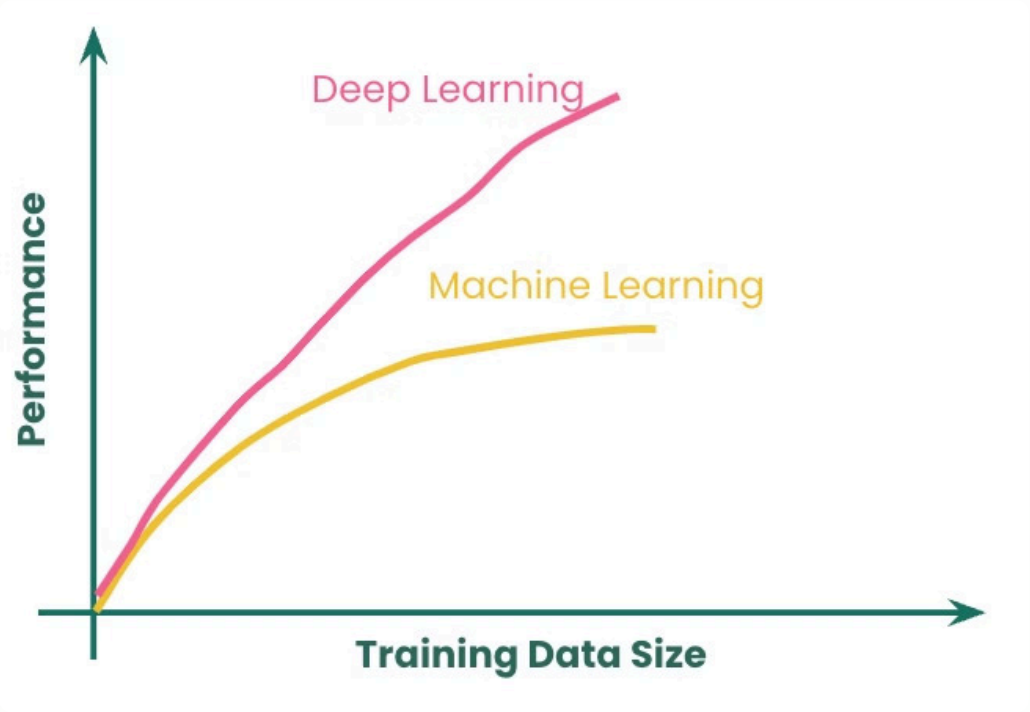


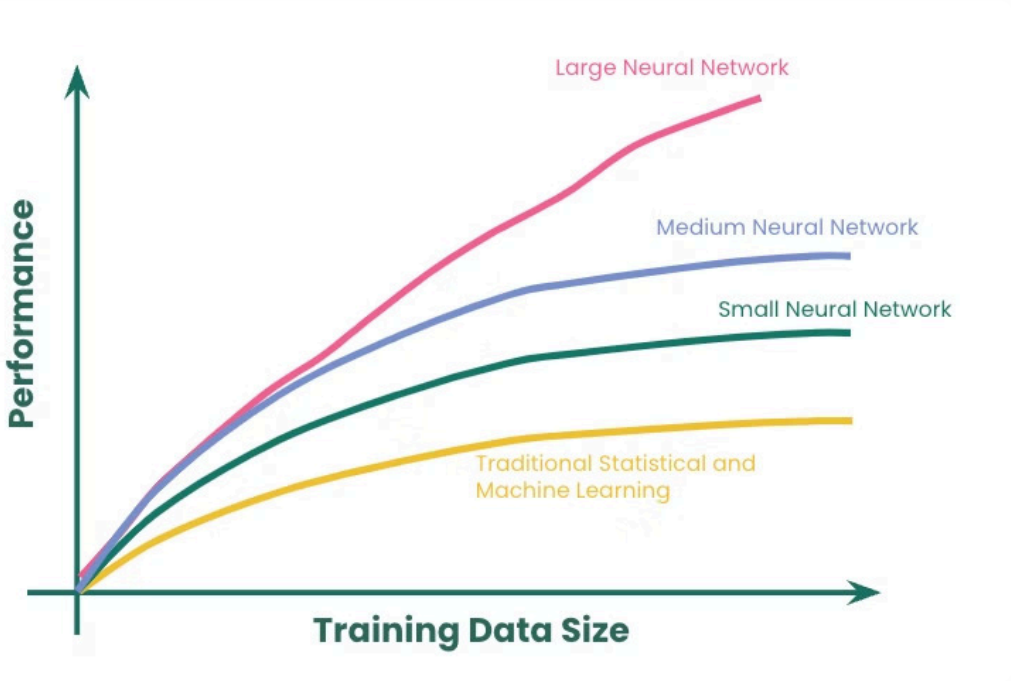
MACHINE LEARNING vs. DEEP LEARNING



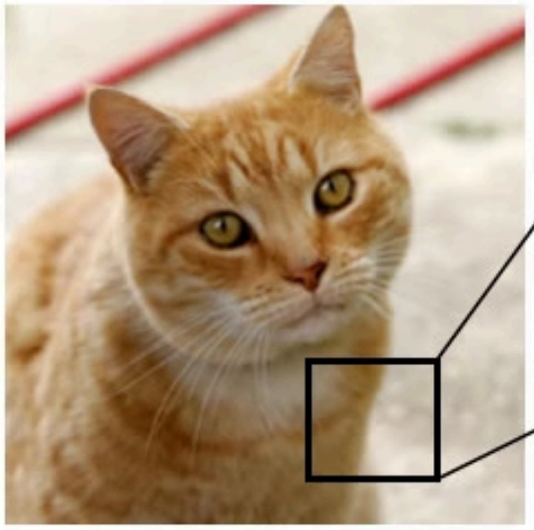
- **Feature Extraction:** Machine Learning requires **manual feature identification** by experts, while Deep Learning automatically learns features from **raw data**.
- **Data Dependency:** Machine Learning works well with **smaller datasets**, while Deep Learning needs **large amounts of data** to excel.
- **Computational Needs:** Machine Learning requires **moderate computing power**, while Deep Learning demands **intensive GPU resources**.
- **Transparency:** Machine Learning models offer **clearer decision-making logic**, while Deep Learning operates as a less interpretable **"black box."**
- **Versatility:** Machine Learning suits **structured data analysis**, while Deep Learning excels at **complex pattern recognition** in images and speech.

MACHINE LEARNING vs. DEEP LEARNING





COMPUTER VISION



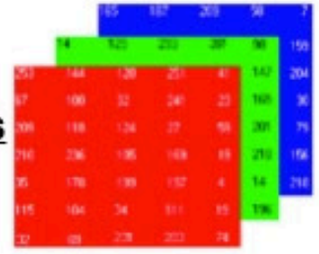
What human observes

```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

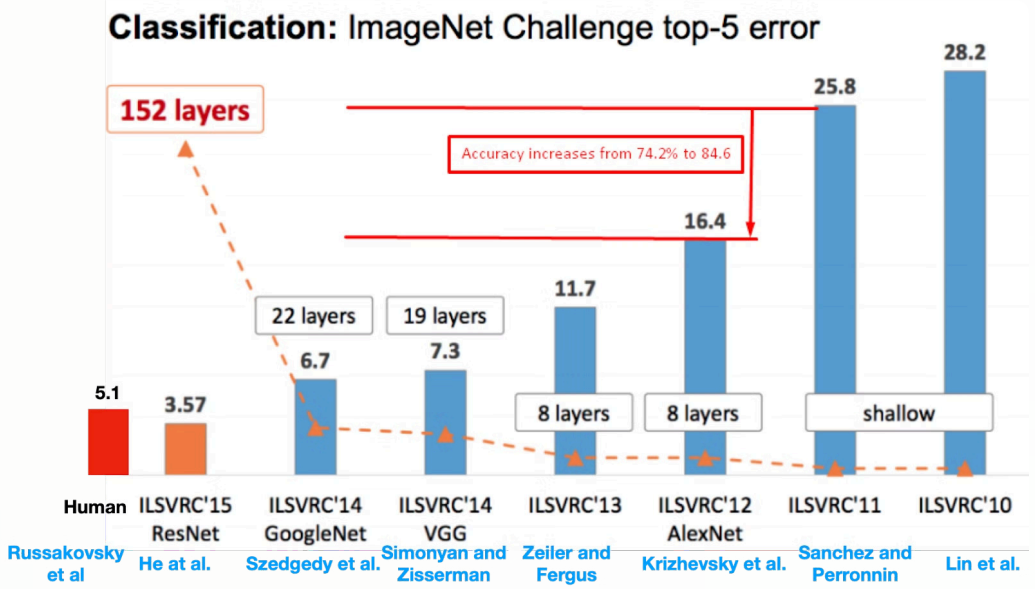
What the computer observes

An image is a matrix of numbers [0,255]

RGB Image has three Channels
RED, Green, Blue



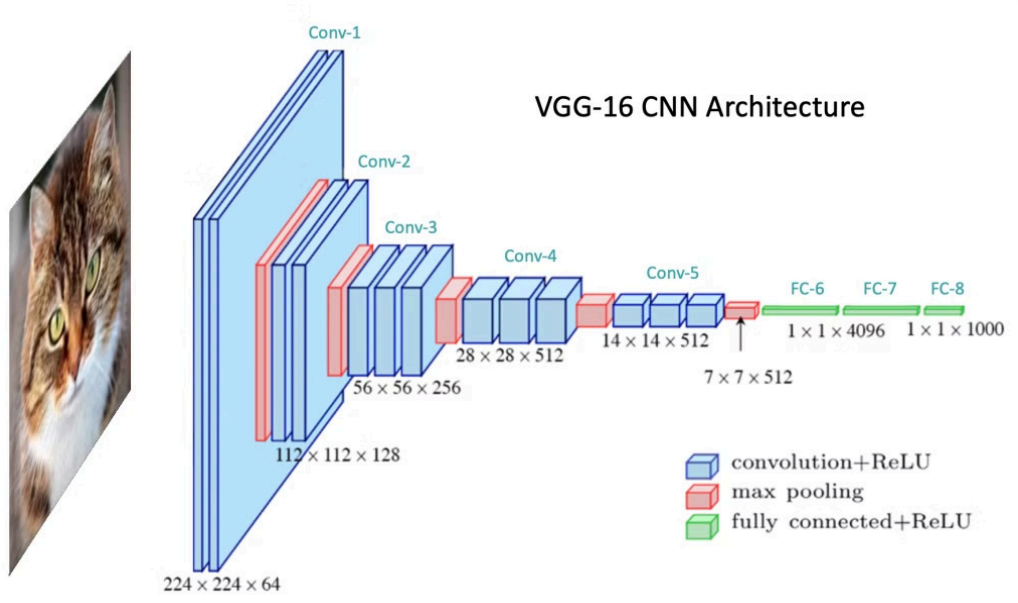
The ImageNet Challenge



- **2010 (Shallow Models):** Top-5 error rate of **28.2%**.
- **2012 (AlexNet):** Introduction of deep learning, **8 layers**, reducing error to **16.4%**.
- **2013 (Zeiler & Fergus):** Improvement to **11.7%**, still using **8 layers**.
- **2014 (VGG):** Increased depth to **19 layers**, achieving **7.3%** error.
- **2014 (GoogLeNet): 22-layer Inception model**, reducing error further to **6.7%**.
- **2015 (ResNet): 152 layers**, achieving a breakthrough with a **3.57%** error rate.
- **Human-Level Performance:** Estimated at **5.1%** error, surpassed by ResNet in 2015.
- **Overall Trend:** Deeper networks improve accuracy significantly, boosting from **74.2%** to **84.6%**.

Convolution Neural Networks

- **Convolution Layers:** Extract spatial patterns using filters
- **Pooling Layers:** Downsample to preserve key features
- **Activation Functions:** Add non-linearity through ReLU and similar functions
- **Fully Connected Layers:** Combine features for final output
- **Parameter Sharing:** Enable efficient, scalable processing
- **Feature Learning:** Progress from basic to complex pattern recognition
- **Regularization:** Prevent overfitting through dropout and normalization
- **Applications:** Power modern computer vision tasks



Deep Learning Evolution: 2010–2015

2010 (NEC-UIUC)

- Handcrafted features like HOG and LBP
- Feature coding and Linear SVM

2012 (SuperVision)

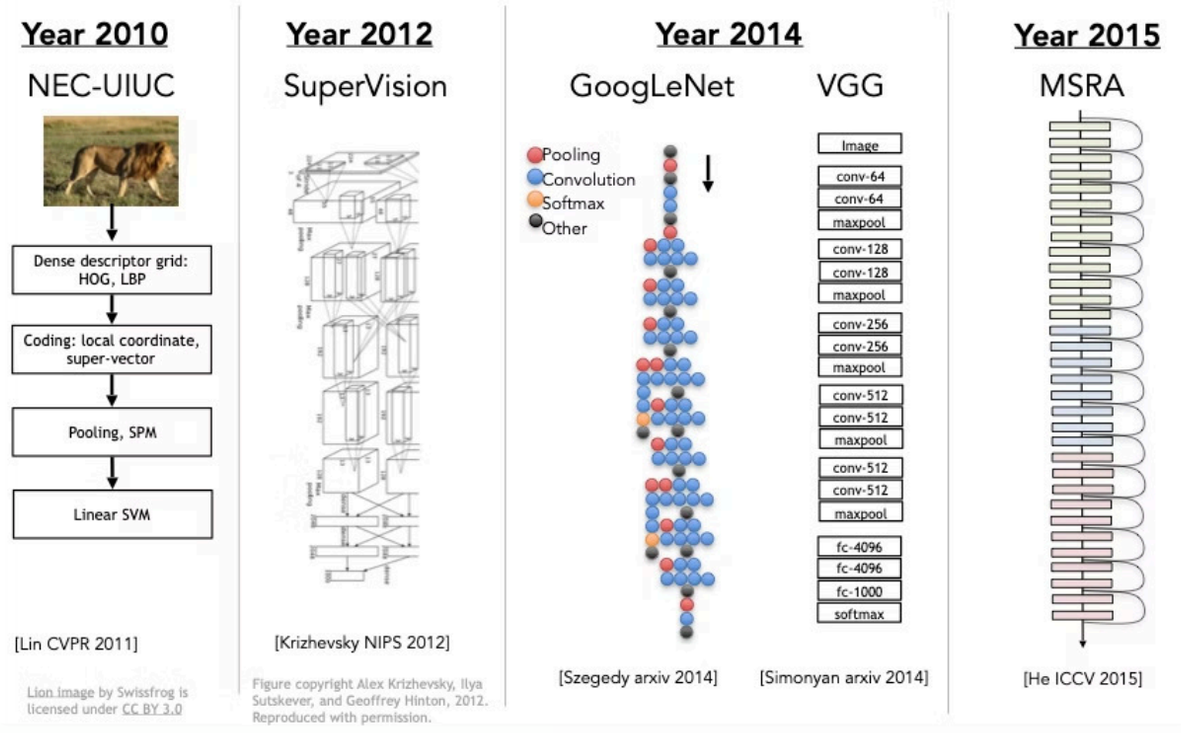
- Convolutional Neural Networks (CNNs)
- **AlexNet** breakthrough on ImageNet
- GPU acceleration for training

2014 (GoogLeNet and VGG)

- **GoogLeNet**: Inception modules
- **VGG**: Very deep networks (up to 19 layers)

2015 (MSRA)

- **ResNet (Residual Networks)**
- Skip connections to address vanishing gradients
- Enabled extremely deep networks (152 layers)



Computer Vision Tasks

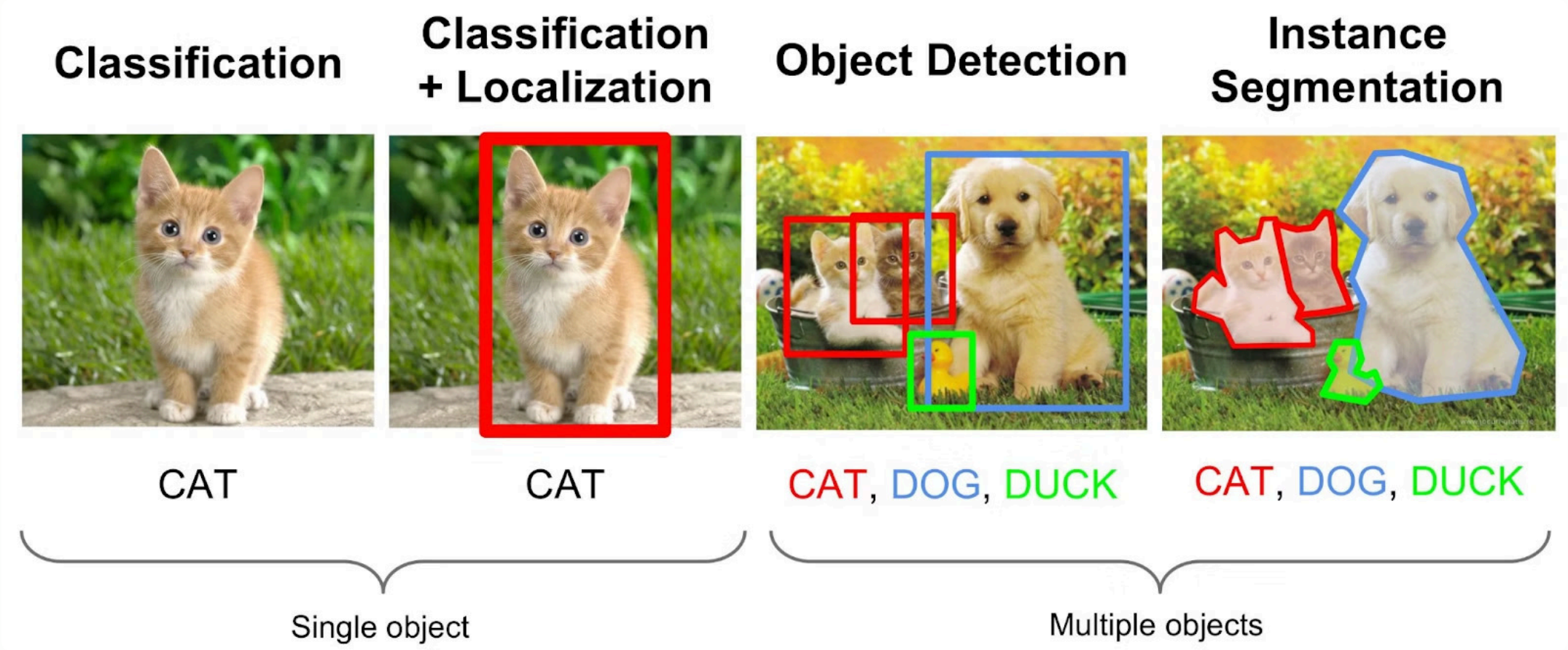
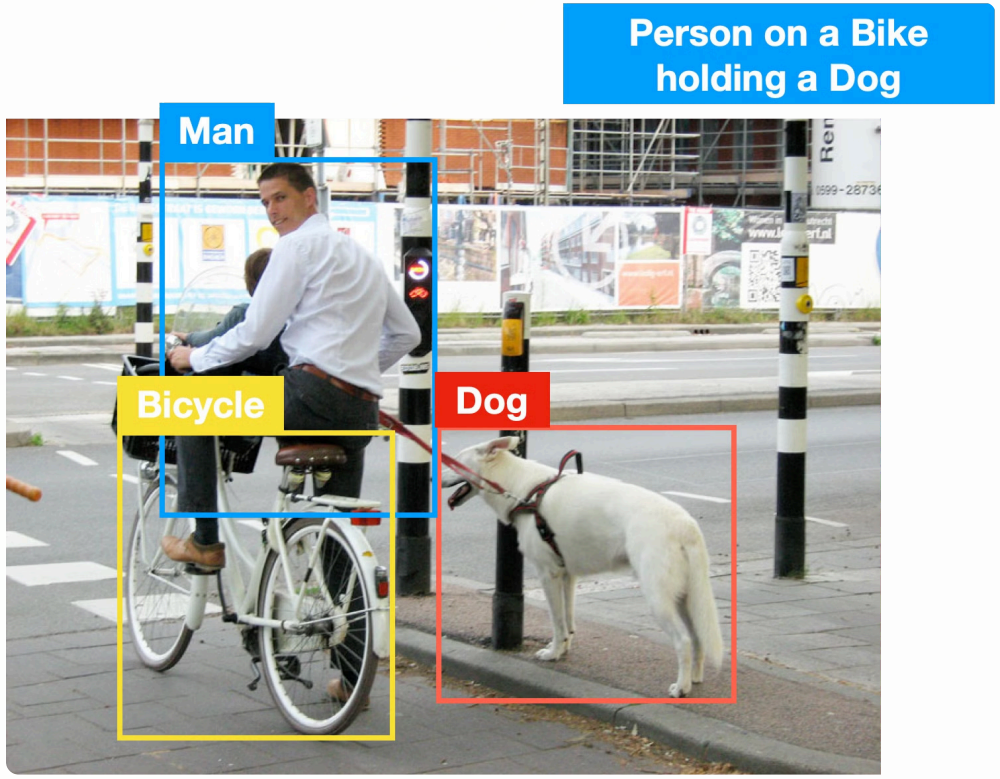


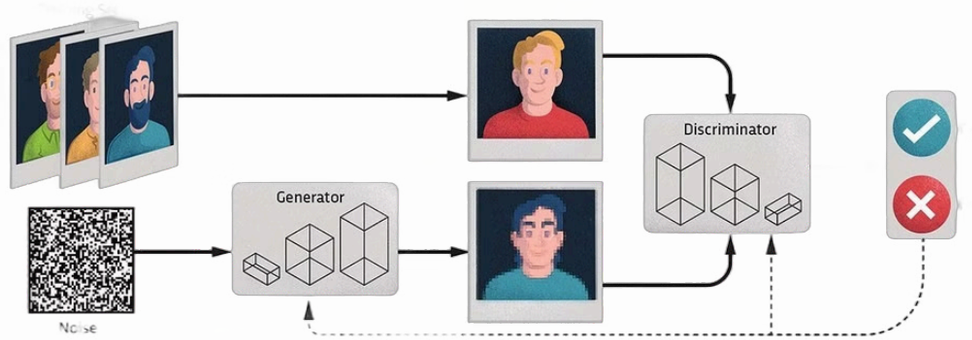
Image Understanding & Generation

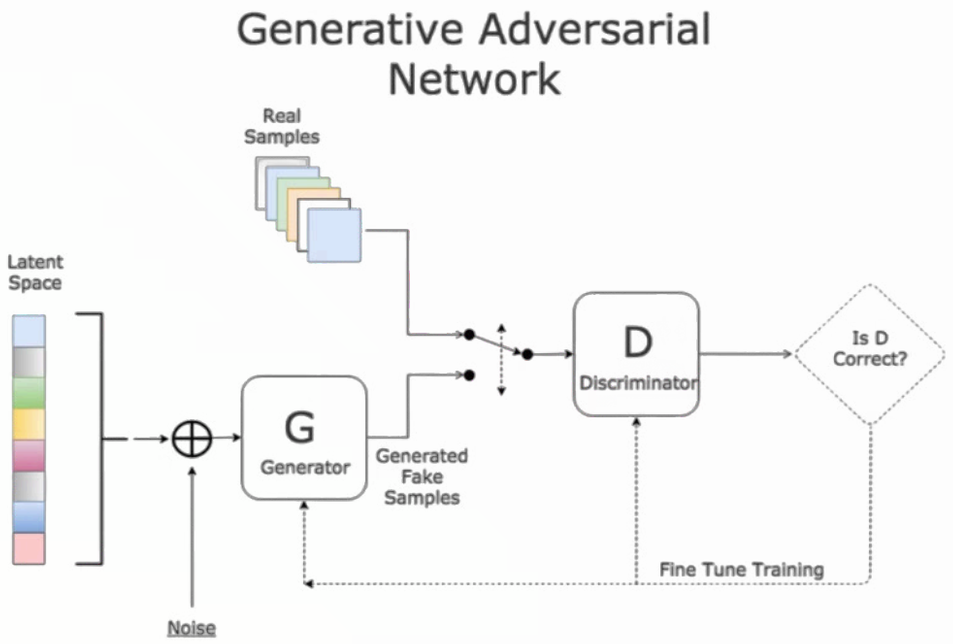
- **Feature Extraction (CNN):**
- Convolutional Networks identify visual patterns in images
- **Sequence Generation (RNN):**
- Recurrent Networks generate descriptive captions
- **End-to-End Architecture:**
- CNNs encode images, RNNs decode into language
- **Training Process:**
- Joint training with image-caption pairs
- Optimized through cross-entropy loss
- **Challenges:**
- Large training datasets required
- Complex visual relationships remain difficult



Generative Adversarial Networks (GANs)

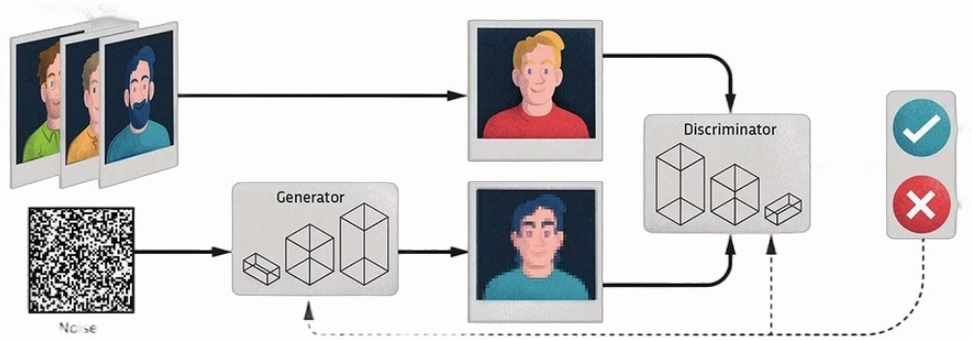
- **Introduced by:** Ian Goodfellow (2014)
- **Key Idea:** Two neural networks (Generator & Discriminator) compete in a zero-sum game.
  - **Generator:** Creates fake data resembling the real dataset.
  - **Discriminator:** Distinguishes between real and fake data.
- **Training Objective:**
  - Generator improves to "fool" the discriminator.
  - Discriminator improves to better detect fakes.
- **Applications:**
  - Image generation (e.g., deepfake creation).
  - Data augmentation for ML models.
  - Super-resolution and style transfer.
- **Challenges:** Training instability, mode collapse, and balancing Generator-Discriminator learning.

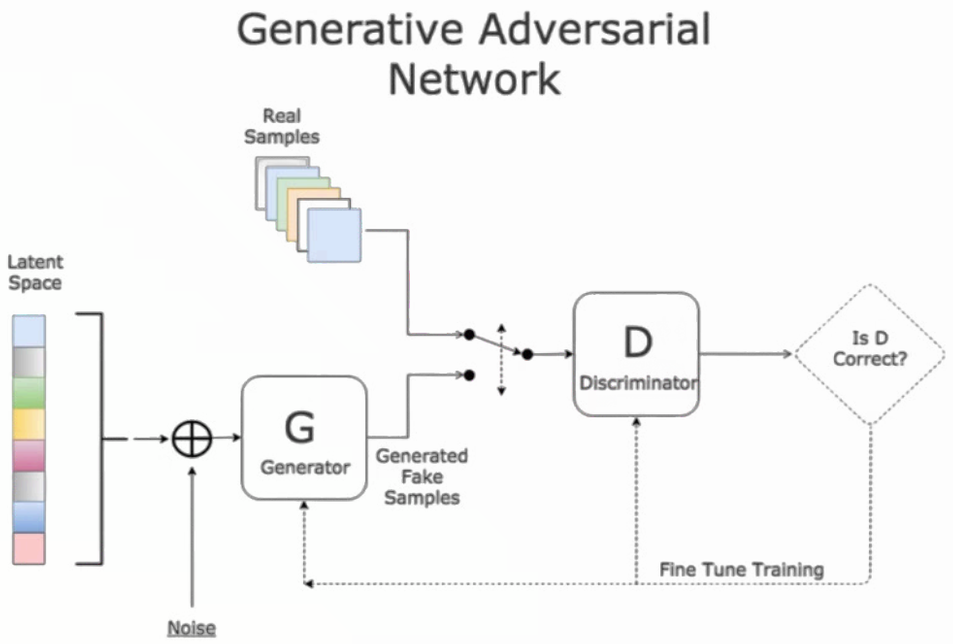




Generative Adversarial Networks (GANs)

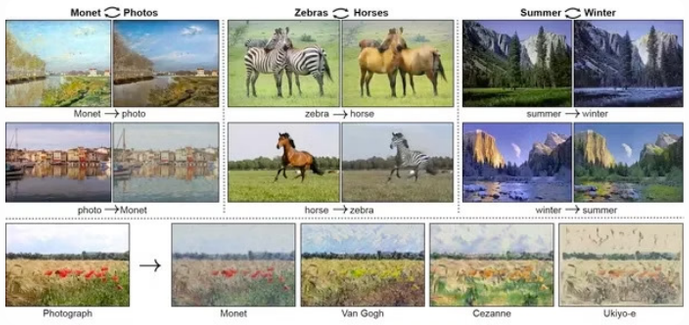
- **Components of GAN:**
  - **Generator (G):** Creates fake data from random noise (z).
  - **Discriminator (D):** Classifies data as real or fake.
- **Training Process:**
- **Step 1:** Generator produces fake samples.
- **Step 2:** Discriminator evaluates real vs. fake samples.
- **Step 3:** Loss is calculated:
  - **Generator Loss:** *minimize* **log(1-D(G(z)))**
  - **Discriminator Loss:** *maximize* **log(D(x)) + log(1-D(G(z)))**
  - D(x) and G(z) are probabilities of real and fake data.
- **Step 4:** Backpropagation updates G and D.
- **Goal:**
- Generator improves to make realistic samples.
- Discriminator sharpens to detect fakes.
- **Output:** Synthetic data indistinguishable from real data.

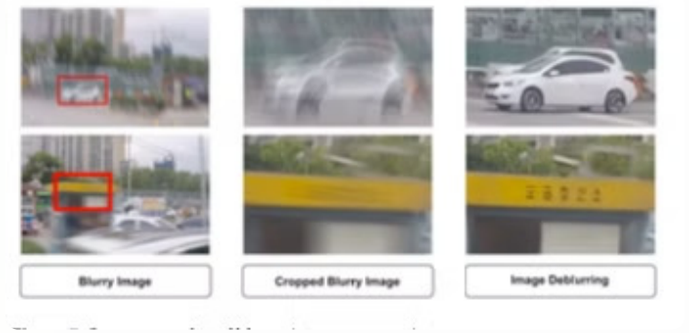




Generative Adversarial Networks Applications

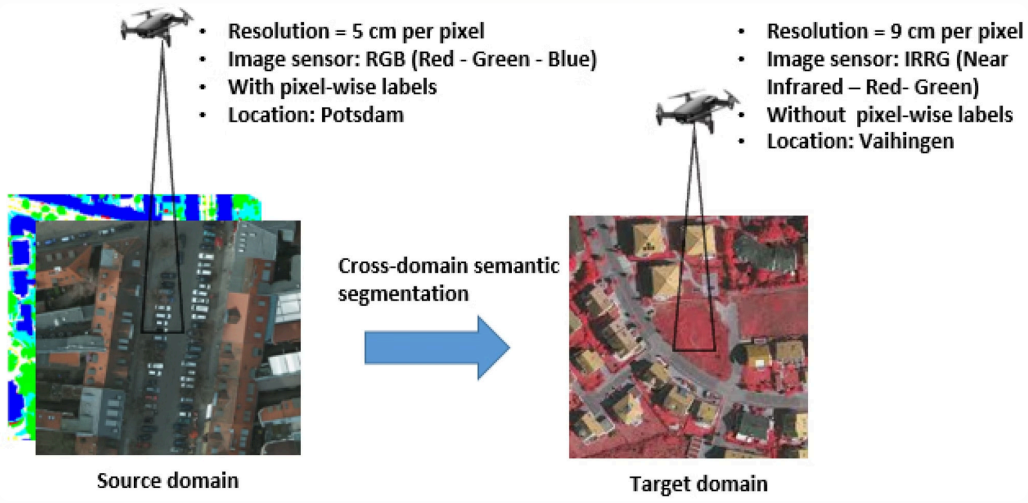
- **Image Generation:** Generate realistic images (e.g., DeepFakes, art creation).
- **Data Augmentation:** Create synthetic data for training ML models.
- **Super-Resolution:** Enhance image resolution (e.g., blurry to sharp images).
- **Style Transfer:** Combine styles of two images (e.g., turning photos into paintings).
- **Text-to-Image Synthesis:** Generate images from textual descriptions.
- **Medical Imaging:** Generate synthetic scans for rare conditions, anomaly detection.
- **Gaming and Virtual Reality:** Generate realistic environments and characters.
- **Domain Adaptation:** Translate data between domains (e.g., summer to winter scenes).

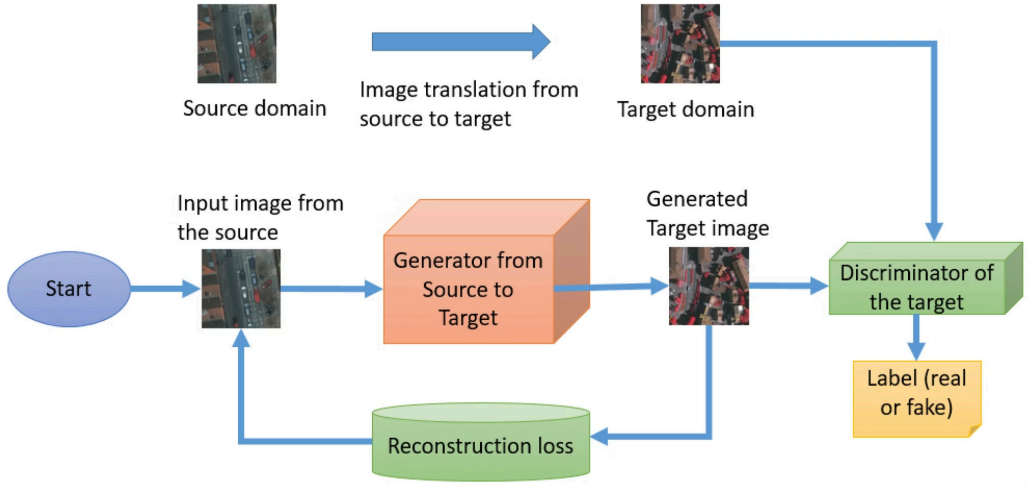




Domain Adaptation using GANs

- **Problem:**
  - Semantic segmentation models fail across domains due to **domain shift** (e.g., sensor type, resolution).
  - Example: Accuracy drops from 85% (Potsdam) to 35% (Vaihingen).
- **Proposed Solution:**
- **GAN-based Unsupervised Domain Adaptation:**
  - Translate images from the **source domain** (Potsdam) to mimic the **target domain** (Vaihingen).
  - Fine-tune segmentation models with translated data.
- **Key Contributions:**
  - Addresses domain shift **without paired datasets** or manual labeling.
  - Improves average segmentation accuracy from **35% to 52%**.
  - Significantly enhances accuracy for classes affected by sensor variation (e.g., **buildings**: 23% → 71%).
- **Efficiency:**
  - Total training time: ~7-10 hours.
  - Practical for real-world aerial imagery scenarios.
- **Impact:**
  - Enables portability of segmentation models across domains with minimal cost.
  - Demonstrates GANs' potential in aerial image analysis.

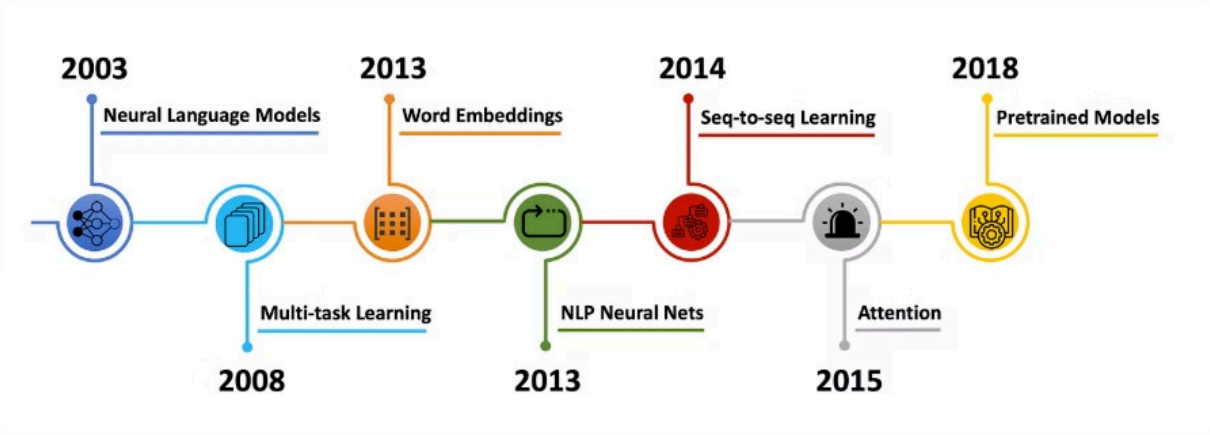




- **Reference:**
  - Bilel Benjdira,Yakoub Bazi,Anis Koubaa, and Kais Ouni, "***Unsupervised Domain Adaptation Using Generative Adversarial Networks for Semantic Segmentation of Aerial Images***", MDPI Remote Sensing, 2019.

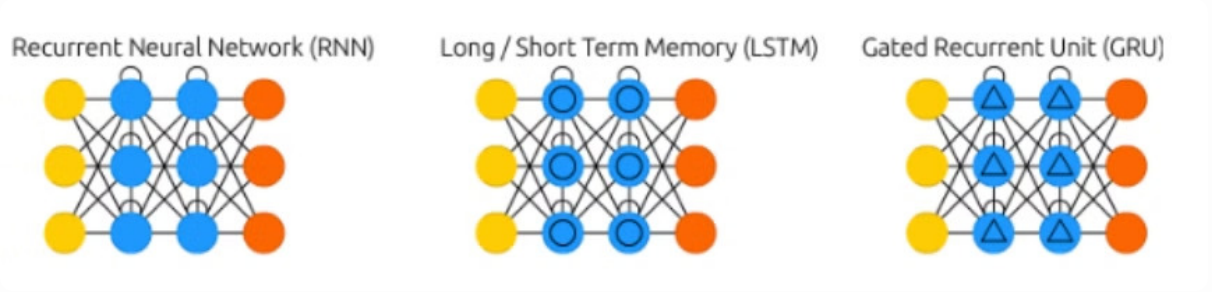
Natural Language Processing (NLP) Evolution

- **2003:** Introduction of **Neural Language Models** for NLP.
- **2008:** Emergence of **Multi-task Learning** to leverage shared representations across tasks.
- **2013:** Development of **Word Embeddings** (e.g., Word2Vec) for vectorizing words.
- **2014:** Introduction of **NLP Neural Nets** and **Seq-to-Seq Learning** for tasks like machine translation.
- **2015:** Use of **Attention Mechanisms** to focus on relevant parts of the input.
- **2018:** Popularization of **Pretrained Models** (e.g., BERT, GPT) revolutionizing NLP.



Natural Language Processing (NLP) Models

- **Recurrent Neural Networks (RNN):**
  - Captures sequential data patterns.
  - Suffers from **vanishing gradients**, limiting long-term dependencies.
- **Long Short-Term Memory (LSTM):**
  - Enhances RNN with **memory cells** to store long-term dependencies.
  - Uses **gates** (input, forget, output) to control information flow.
- **Gated Recurrent Unit (GRU):**
  - Simplified version of LSTM.
  - Combines **update** and **reset gates** for faster training with fewer parameters.



Key Characteristics:

- **RNN:** Basic sequential learning.
- **LSTM:** Handles long-term dependencies effectively.
- **GRU:** Efficient and lightweight alternative to LSTM.

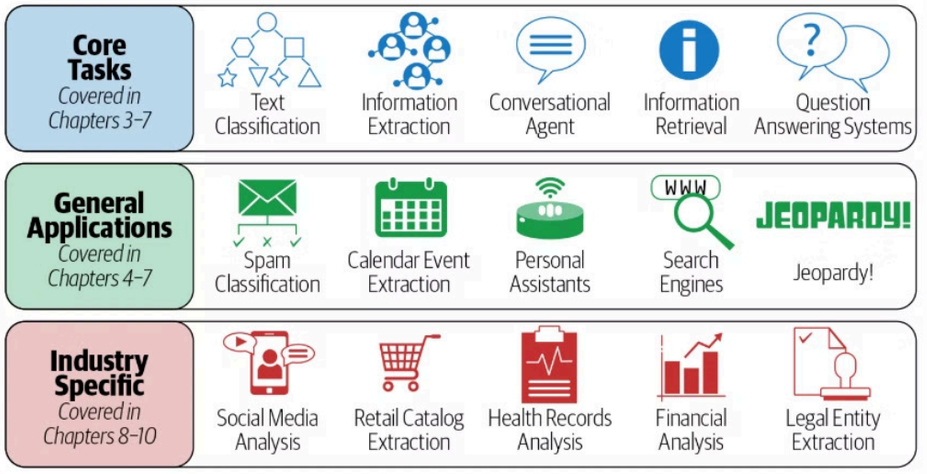
Natural Language Processing (NLP) Applications

Core Tasks

- **Text Classification:** Categorizing text (e.g., spam detection, sentiment analysis).
- **Information Extraction:** Extracting structured data from unstructured text (e.g., names, dates).
- **Conversational Agents:** Building chatbots and virtual assistants.
- **Information Retrieval:** Finding relevant documents (e.g., search engines).
- **Question Answering:** Answering queries from knowledge bases or documents.

General Applications

- **Spam Classification:** Filtering emails or messages into spam/non-spam.
- **Calendar Event Extraction:** Identifying and creating events from text.
- **Personal Assistants:** Siri, Alexa, and Google Assistant for daily tasks.
- **Search Engines:** NLP-driven queries to improve search results.
- **Jeopardy!:** Answering context-sensitive trivia questions.



Industry-Specific Applications

- **Social Media Analysis:** Sentiment analysis and trend detection on platforms like Twitter.
- **Retail Catalog Extraction:** Organizing product details for e-commerce.
- **Health Records Analysis:** Extracting insights from medical records for diagnosis.
- **Financial Analysis:** Parsing financial reports for decision-making.
- **Legal Entity Extraction:** Identifying entities in contracts and legal documents.

Attention is All You Need!

Attention is All You Need!

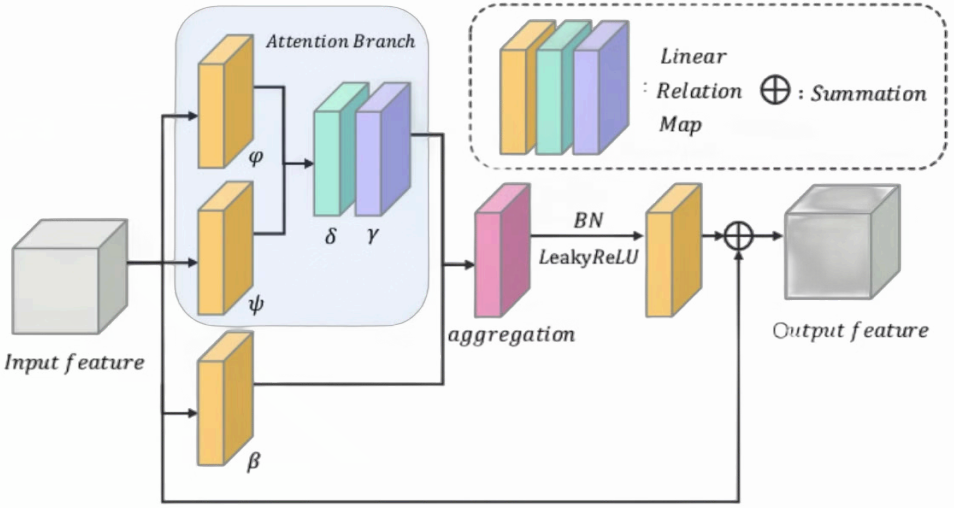
- **Key Paper Information:**
  - **Authors:** Ashish Vaswani et al.
  - **Published:** 2017.
  - **Journal:** arXiv preprint (arXiv:1706.03762).
  - **Citations:** Over **155,142** citations (as of 2024).
- **Core Contribution:**
  - Introduced the **Transformer architecture**.
  - Replaced RNNs and CNNs with a model based entirely on **self-attention mechanisms**.
  - Basis for modern **Generative AI** and **Large Language Models (LLMs)** (e.g., GPT, BERT).
- **Citation Growth:**
  - Rapid increase in citations, reflecting its transformative impact on NLP and AI.
  - **2023 alone:** Over **47,782 citations**.

- **Significance:**
  - Marked a paradigm shift in machine learning.
  - Empowered advancements in NLP tasks like **translation, summarization, and text generation**.
  - Enabled scaling of models to unprecedented levels.

The New NLP Era: Self-Attention and Transformers

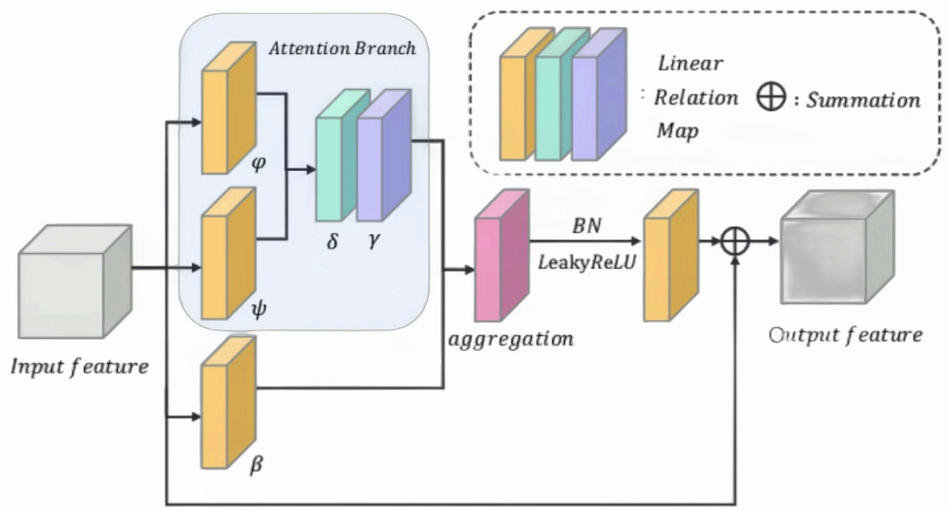
Self-Attention: Key Innovation

- Enables the model to focus on **relevant parts** of the input sequence.
- Computes relationships between all tokens, assigning **attention weights**.
- Solves the limitation of RNNs and LSTMs by capturing **long-range dependencies** efficiently.



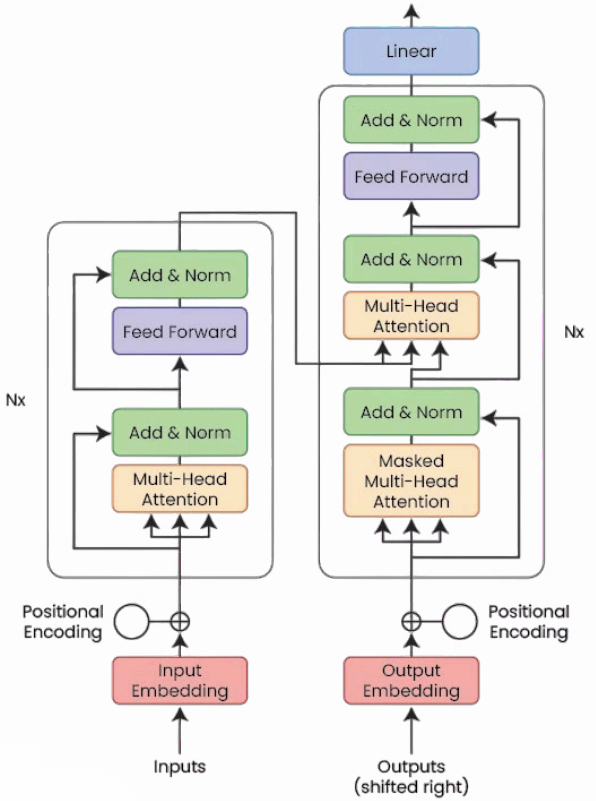
The New NLP Era: Self-Attention and Transformers

- **What is Self-Attention?**
  - Captures relationships between all input tokens in a sequence.
  - Assigns **attention weights** to focus on the most relevant parts of the input.
- **Key Steps:**
  - **Input Features:** Each token is embedded into a vector representation.
  - **Attention Branch:** Computes scaled attention scores (ϕ, ψ, δ, γ).
  - **Aggregation:** Combines weighted features to highlight important tokens.
  - **Output:** Produces refined representations capturing global context.
- **Advantages of Self-Attention:**
  - Models **long-range dependencies** effectively.
  - Parallel computation enables **scalability and efficiency**.
  - Forms the backbone of **Transformers** for modern NLP tasks.



Transformer Architecture: Revolutionizing NLP

- **What is a Transformer?**
  - A model architecture based entirely on **self-attention mechanisms**.
  - Replaces sequential processing (RNNs) with **parallel computation** for scalability.
- **Key Components (Using Visual):**
- **Encoder:**
  - Processes the input sequence.
  - Uses multi-head self-attention to capture relationships between tokens.
- **Decoder:**
  - Generates the output sequence.
  - Incorporates self-attention and attention to the encoder's output.
- **Add & Norm:** Ensures stable training.
- **Feed-Forward Layers:** Applies transformations to enhance feature representation.
- **Positional Encoding:**
  - Adds order information to the sequence since attention mechanisms are permutation-invariant.

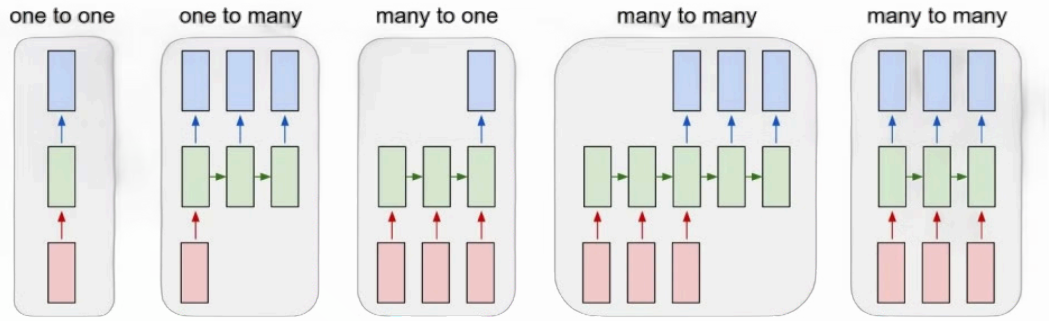


How did we get to Self-Attention?

Sequential Data

Traditional Sequential Models

- **One-to-One:** Single input → Single output (e.g., image classification).
- **One-to-Many:** Single input → Sequence output (e.g., image captioning).
- **Many-to-One:** Sequence input → Single output (e.g., sentiment analysis).
- **Many-to-Many (Aligned):** Sequence input → Sequence output (e.g., video frame labeling).
- **Many-to-Many (Unaligned):** Sequence input → Sequence output with different lengths (e.g., machine translation).



Origin of RNN (1990)

COGNITIVE SCIENCE 14, 179–211 (1990)

Finding Structure in Time

JEFFREY L. ELMAN
University of California, San Diego

Time underlies many interesting human behaviors. Thus, the question of how to represent time in connectionist models is very important. One approach is to represent time implicitly by its effects on processing rather than explicitly (as in a spatial representation). The current report develops a proposal along these lines first described by Jordan (1986) which involves the use of recurrent links in order to provide networks with a dynamic memory. In this approach, hidden unit patterns are fed back to themselves; the internal representations which develop thus reflect task demands in the context of prior internal states. A set of simulations is reported which range from relatively simple problems (temporal version of XOR) to discovering syntactic/semantic features for words. The networks are able to learn interesting internal representations which incorporate task demands with memory demands; indeed, in this approach the notion of memory is inextricably bound up with task processing. These representations reveal a rich structure, which allows them to be highly context-dependent, while also expressing generalizations across classes of items. These representations suggest a method for representing lexical categories and the type/token distinction.

http://psych.colorado.edu/~kimlab/Elman1990.pdf

Key Concepts in Elman's "Finding Structure in Time"

- **Temporal Representation in Neural Networks**
  - Represents time-based patterns in neural networks
  - Essential for sequential data processing
- **Recurrent Neural Networks (RNNs)**
  - Networks with directed temporal connections
  - Foundation for modern language models
- **Dynamic Memory in Networks**
  - Enables sequential information processing
  - Key component for context understanding in LLMs
- **Reference**
  - http://psych.colorado.edu/~kimlab/Elman1990.pdf

Origin of RNN (1990)

COGNITIVE SCIENCE 14, 179–211 (1990)

Finding Structure in Time

JEFFREY L. ELMAN
University of California, San Diego

Time underlies many interesting human behaviors. Thus, the question of how to represent time in connectionist models is very important. One approach is to represent time implicitly by its effects on processing rather than explicitly (as in a spatial representation). The current report develops a proposal along these lines first described by Jordan (1986) which involves the use of recurrent links in order to provide networks with a dynamic memory. In this approach, hidden unit patterns are fed back to themselves; the internal representations which develop thus reflect task demands in the context of prior internal states. A set of simulations is reported which range from relatively simple problems (temporal version of XOR) to discovering syntactic/semantic features for words. The networks are able to learn interesting internal representations which incorporate task demands with memory demands; indeed, in this approach the notion of memory is inextricably bound up with task processing. These representations reveal a rich structure, which allows them to be highly context-dependent, while also expressing generalizations across classes of items. These representations suggest a method for representing lexical categories and the type/token distinction.

http://psych.colorado.edu/~kimlab/Elman1990.pdf

Elman's Impact on Large Language Models

- **Internal Representation of Sequences**
  - Pioneered understanding of temporal sequence processing
  - Forms foundation for modern LLM text processing
- **Learning and Generalization**
  - Established how networks learn temporal patterns
  - Enabled LLMs to generalize across text patterns
- **NLP Applications**
  - Research extended beyond language to core NLP concepts
  - Directly influenced modern language model architecture

Origin of RNN (1990)

COGNITIVE SCIENCE **14,** 179–211 (1990)

Finding Structure in Time

JEFFREY L. ELMAN
University of California, San Diego

Time underlies many interesting human behaviors. Thus, the question of how to represent time in connectionist models is very important. One approach is to represent time implicitly by its effects on processing rather than explicitly (as in a spatial representation). The current report develops a proposal along these lines first described by Jordan (1986) which involves the use of recurrent links in order to provide networks with a dynamic memory. In this approach, hidden unit patterns are fed back to themselves; the internal representations which develop thus reflect task demands in the context of prior internal states. A set of simulations is reported which range from relatively simple problems (temporal version of XOR) to discovering syntactic/semantic features for words. The networks are able to learn interesting internal representations which incorporate task demands with memory demands; indeed, in this approach the notion of memory is inextricably bound up with task processing. These representations reveal a rich structure, which allows them to be highly context-dependent, while also expressing generalizations across classes of items. These representations suggest a method for representing lexical categories and the type/token distinction.

http://psych.colorado.edu/~kimlab/Elman1990.pdf

NETWORKS WITH MEMORY

The spatial representation of time described above treats time as an explicit part of the input. There is another, very different possibility: Allow time to be represented by the effect it has on processing. This means giving the processing system dynamic properties that are responsive to temporal sequences. In short, the network must be given memory.

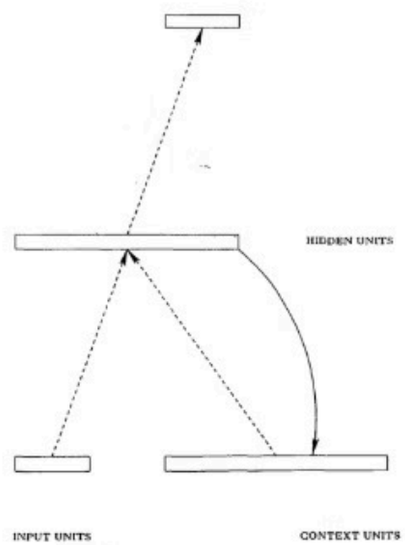


Figure 2. A simple recurrent network in which activations are copied from hidden layer to context layer on a one-for-one basis, with fixed weight of 1.0. Dotted lines represent trainable connections.

Origin of RNN (1990)

COGNITIVE SCIENCE **14,** 179–211 (1990)

Finding Structure in Time

JEFFREY L. ELMAN
University of California, San Diego

Time underlies many interesting human behaviors. Thus, the question of how to represent time in connectionist models is very important. One approach is to represent time implicitly by its effects on processing rather than explicitly (as in a spatial representation). The current report develops a proposal along these lines first described by Jordan (1986) which involves the use of recurrent links in order to provide networks with a dynamic memory. In this approach, hidden unit patterns are fed back to themselves; the internal representations which develop thus reflect task demands in the context of prior internal states. A set of simulations is reported which range from relatively simple problems (temporal version of XOR) to discovering syntactic/semantic features for words. The networks are able to learn interesting internal representations which incorporate task demands with memory demands; indeed, in this approach the notion of memory is inextricably bound up with task processing. These representations reveal a rich structure, which allows them to be highly context-dependent, while also expressing generalizations across classes of items. These representations suggest a method for representing lexical categories and the type/token distinction.

http://psych.colorado.edu/~kimlab/Elman1990.pdf

DISCOVERING THE NOTION "WORD"

It is taken for granted that learning a language involves (among many other things) learning the sounds of that language, as well as the morphemes and words. Many theories of acquisition depend crucially upon such primitive types as word, or morpheme, or more abstract categories as noun, verb, or phrase (e.g., Berwick & Weinberg, 1984; Pinker, 1984). Rarely is it asked how a language learner knows when to begin or why these entities exist. These notions are often assumed to be innate.

Yet, in fact, there is considerable debate among linguists and psycholinguists about what representations are used in language. Although it is commonplace to speak of basic units such as "phoneme," "morpheme," and "word," these constructs have no clear and uncontroversial definition.

TABLE 5
Fragment of Training Sequences for Sentence Simulation

Input	Output
00000000000000000000000000010 (*woman*)	00000000000000000000000010000 (*smash*)
00000000000000000000000010000 (*smash*)	00000000000000000001000000000 (*plate*)
00000000000000000001000000000 (*plate*)	00000100000000000000000000000 (*cat*)
00000100000000000000000000000 (*cat*)	00000000000000000100000000000 (*move*)
00000000000000000100000000000 (*move*)	00000000000000001000000000000 (*man*)
00000000000000001000000000000 (*man*)	00010000000000000000000000000 (*break*)
00010000000000000000000000000 (*break*)	00001000000000000000000000000 (*car*)
00001[illegible]	[illegible]y)
01000[illegible]	[illegible]ve)
00000[illegible]	[illegible]l)
00000[illegible]	[illegible]?)
00000[illegible]	[illegible]ead)
00100000000000000000000000000 (*bread*)	00000001000000000000000000000 (*dog*)
00000000100000000000000000000 (*dog*)	00000000000000000100000000000 (*move*)
00000000000000000100000000000 (*move*)	00000000000000001000000000000 (*mouse*)
00000000000000001000000000000 (*mouse*)	00000000000000001000000000000 (*mouse*)
00000000000000001000000000000 (*mouse*)	00000000000000000100000000000 (*move*)
00000000000000000100000000000 (*move*)	10000000000000000000000000000 (*book*)
10000000000000000000000000000 (*book*)	00000000000000100000000000000 (*lion*)

One-Hot Encoding/Embedding

Origin of RNN (1990)

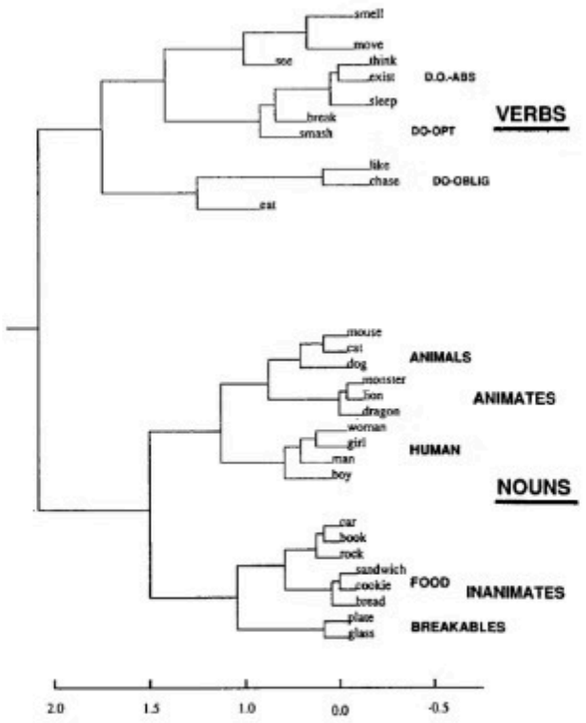


Figure 7. Hierarchical cluster diagram of hidden unit activation vectors in simple sentence prediction task. Labels indicate the inputs which produced the hidden unit vectors; inputs were presented in context, and the hidden unit vectors averaged across multiple contexts.

http://psych.colorado.edu/~kimlab/Elman1990.pdf

COGNITIVE SCIENCE **14,** 179-211 (1990)

Finding Structure in Time

JEFFREY L. ELMAN
University of California, San Diego

the output layer. These were ignored. Instead, the hidden unit activations for each word + context input were saved, resulting in 27,354 150-bit vectors. Each word occurs many times, in different contexts. As a first approximation of a word's prototypical or composite representation, all hidden unit activation patterns produced by a given word (in all its contexts) were averaged to yield a single 150-bit vector for each of the 29 unique words in the input stream.[6] (In the next section it will be shown how it is possible to study the internal representations of words in context.) These internal representations were then subject to a hierarchical clustering analysis. Figure 7 shows the resulting tree; this tree reflects the similarity structure of the internal representations these lexical items. Lexical items which have similar properties are grouped together lower in the tree, and clusters of similar words which resemble other clusters are connected higher in the tree.

The network has discovered that there are several major categories of words. One large category corresponds to *verbs;* another category corresponds to *nouns.* The verb category is broken down into groups that *require a direct object,* or are *intransitive,* or where a *direct object is optional.* The noun category is broken into two major groups: *inanimates,* and *animates.* Animates are divided into *human* and *nonhuman;* the nonhuman are divided into *large animals* and *small animals.* Inanimates are broken into *breakable, edibles,* and nouns which appeared as subjects of agentless active verbs.

The network has developed internal representations for the input vectors which reflect facts about the possible sequential ordering of the inputs. The network is not able to predict the precise order of words, but it recognizes that (in this corpus) there is a class of inputs (namely, verbs) that typically follow other inputs (namely, nouns). This knowledge of class behavior is

Origin of RNN (1990)

COGNITIVE SCIENCE **14,** 179–211 (1990)

Finding Structure in Time

JEFFREY L. ELMAN
University of California, San Diego

Time underlies many interesting human behaviors. Thus, the question of how to represent time in connectionist models is very important. One approach is to represent time implicitly by its effects on processing rather than explicitly (as in a spatial representation). The current report develops a proposal along these lines first described by Jordan (1986) which involves the use of recurrent links in order to provide networks with a dynamic memory. In this approach, hidden unit patterns are fed back to themselves; the internal representations which develop thus reflect task demands in the context of prior internal states. A set of simulations is reported which range from relatively simple problems (temporal version of XOR) to discovering syntactic/semantic features for words. The networks are able to learn interesting internal representations which incorporate task demands with memory demands; indeed, in this approach the notion of memory is inextricably bound up with task processing. These representations reveal a rich structure, which allows them to be highly context-dependent, while also expressing generalizations across classes of items. These representations suggest a method for representing lexical categories and the type/token distinction.

http://psych.colorado.edu/~kimlab/Elman1990.pdf

TYPES, TOKENS, AND STRUCTURED REPRESENTATIONS

There has been considerable discussion about the ways in which PDP networks differ from traditional computational models. One apparent difference is that traditional models involve symbolic representations, whereas PDP nets seem, to many people, to be non- or perhaps subsymbolic (Fodor & Pylyshyn, 1988; Smolensky, 1987, 1988). This is a difficult and complex issue, in part because the definition of symbol is problematic. Symbols do many things, and it might be more useful to contrast PDP versus traditional models with regard to the various functions that symbols can serve.

Origin of RNN (1990)

COGNITIVE SCIENCE **14,** 179–211 (1990)

Finding Structure in Time

JEFFREY L. ELMAN
University of California, San Diego

Time underlies many interesting human behaviors. Thus, the question of how to represent time in connectionist models is very important. One approach is to represent time implicitly by its effects on processing rather than explicitly (as in a spatial representation). The current report develops a proposal along these lines first described by Jordan (1986) which involves the use of recurrent links in order to provide networks with a dynamic memory. In this approach, hidden unit patterns are fed back to themselves; the internal representations which develop thus reflect task demands in the context of prior internal states. A set of simulations is reported which range from relatively simple problems (temporal version of XOR) to discovering syntactic/semantic features for words. The networks are able to learn interesting internal representations which incorporate task demands with memory demands; indeed, in this approach the notion of memory is inextricably bound up with task processing. These representations reveal a rich structure, which allows them to be highly context-dependent, while also expressing generalizations across classes of items. These representations suggest a method for representing lexical categories and the type/token distinction.

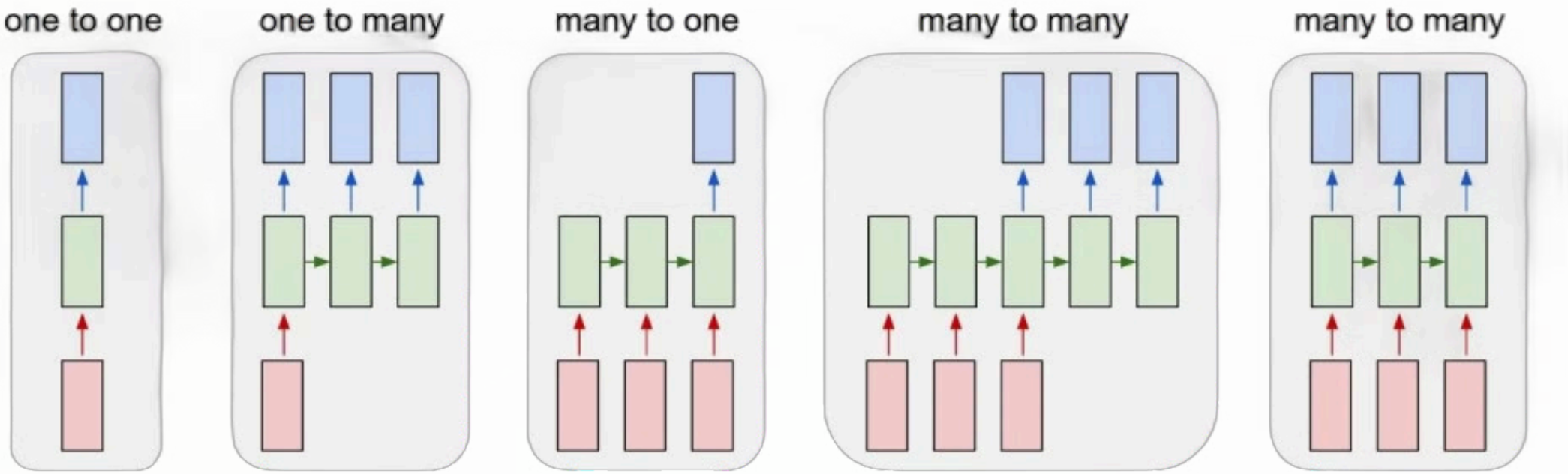
http://psych.colorado.edu/~kimlab/Elman1990.pdf

I would like to thank Jay McClelland, Mike Jordan, Mary Hare, Dave Rumelhart, Mike Mozer, Steve Poteet, David Zipser, and Mark Dolson for many stimulating discussions. I thank McClelland, Jordan, and two anonymous reviewers for helpful critical comments on an earlier draft of this article.

This work was supported by contract N000114-85-K-0076 from the Office of Naval Research and contract DAAB-07-87-C-H027 from Army Avionics, Ft. Monmouth.

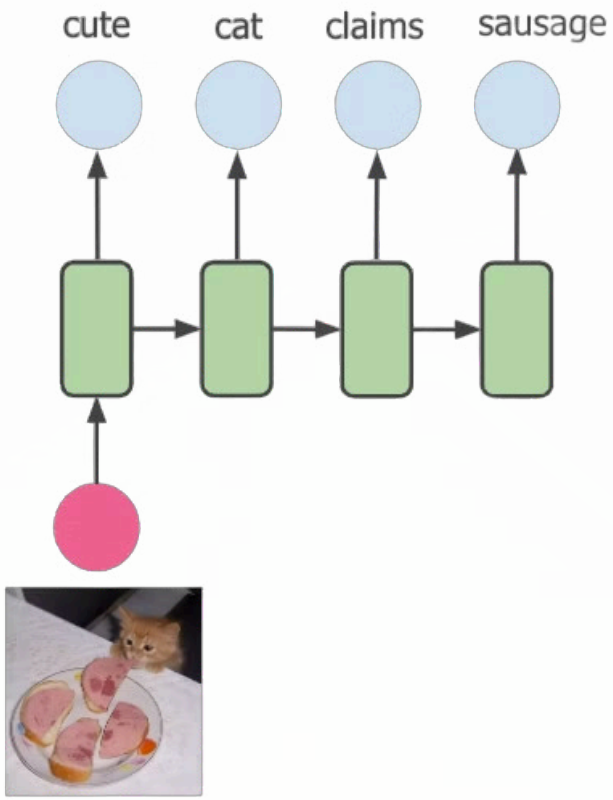
Correspondence and requests for reprints should be sent to Jeffrey L. Elman, Center for Research in Language, C-008, University of California, La Jolla, CA 92093.

Sequential Models

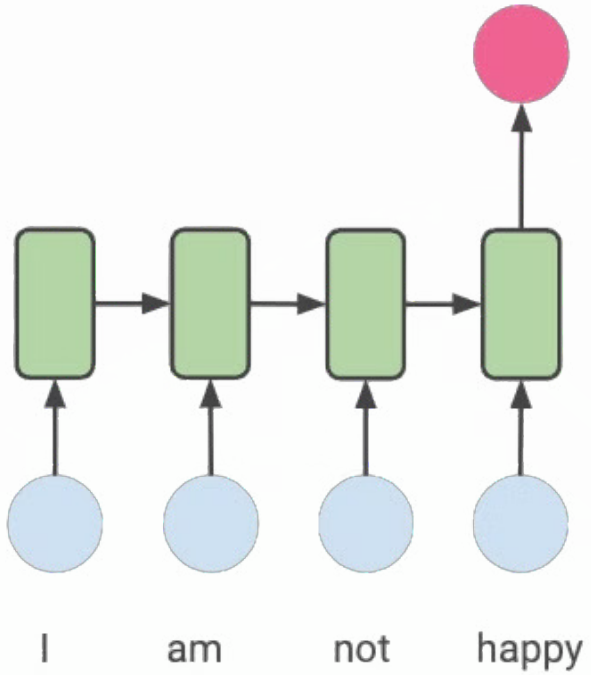


One-to-One | Binary Classification

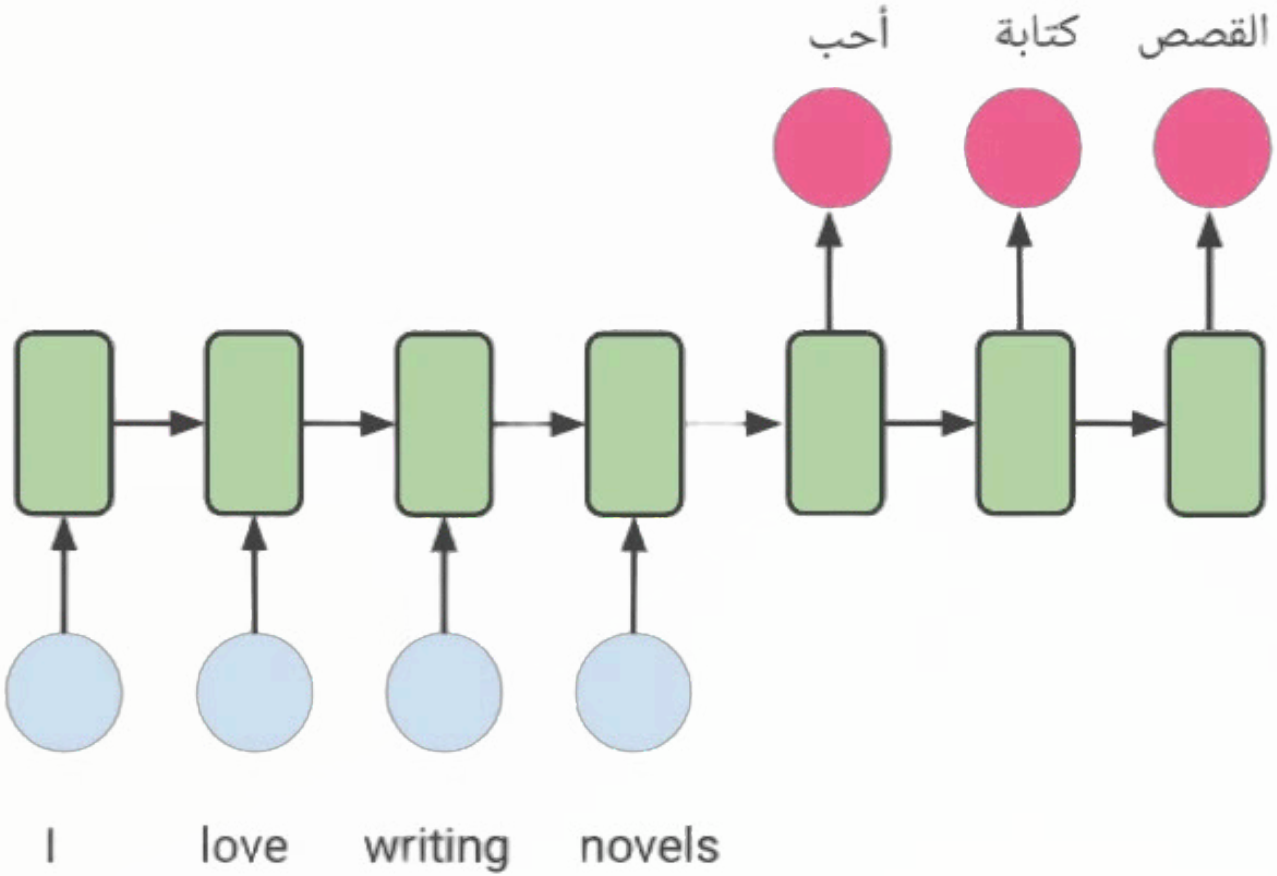
One-to-Many | Image Captioning



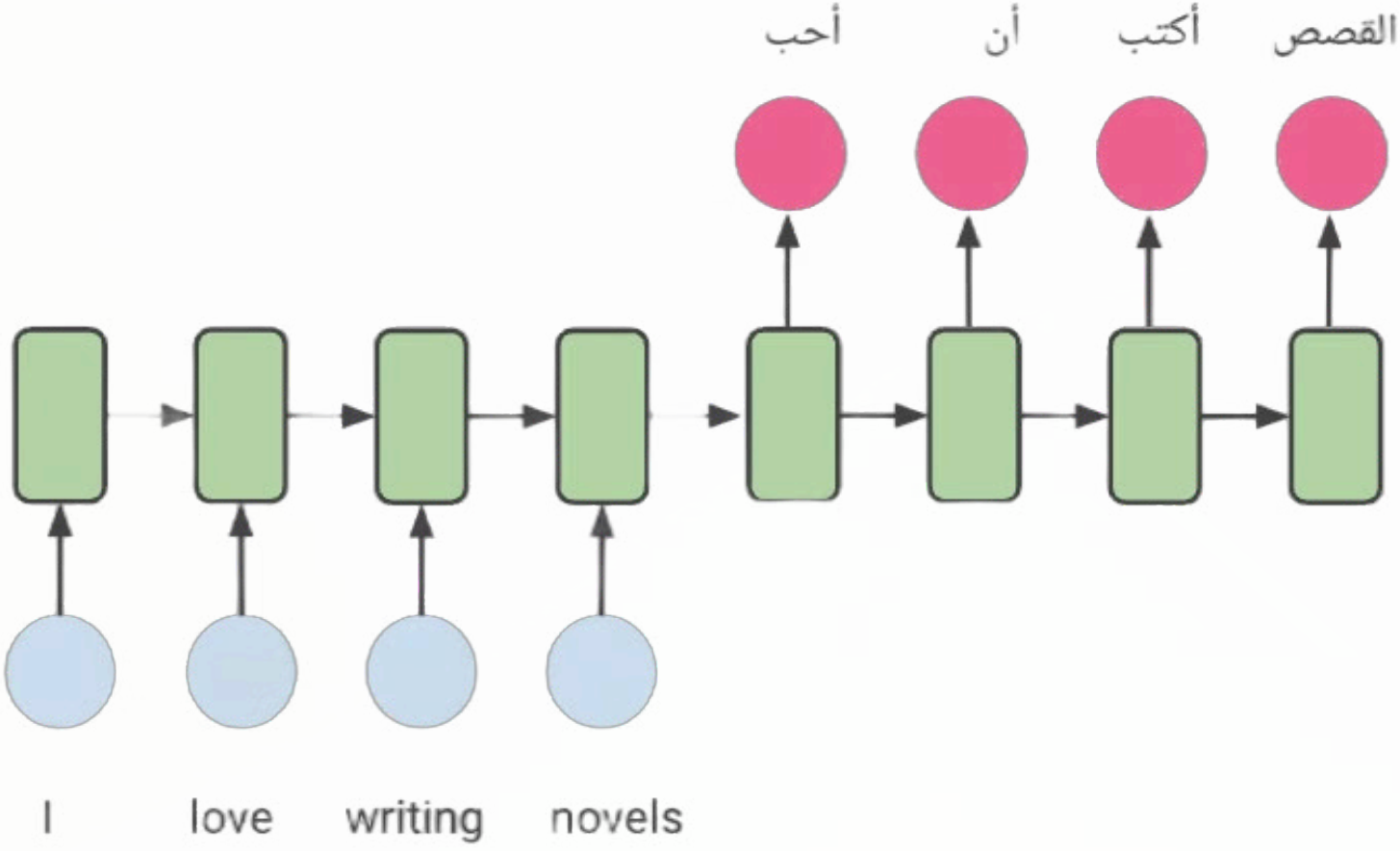
Many-to-One | Sentiment Analysis



Many-to-Many | Machine Translation

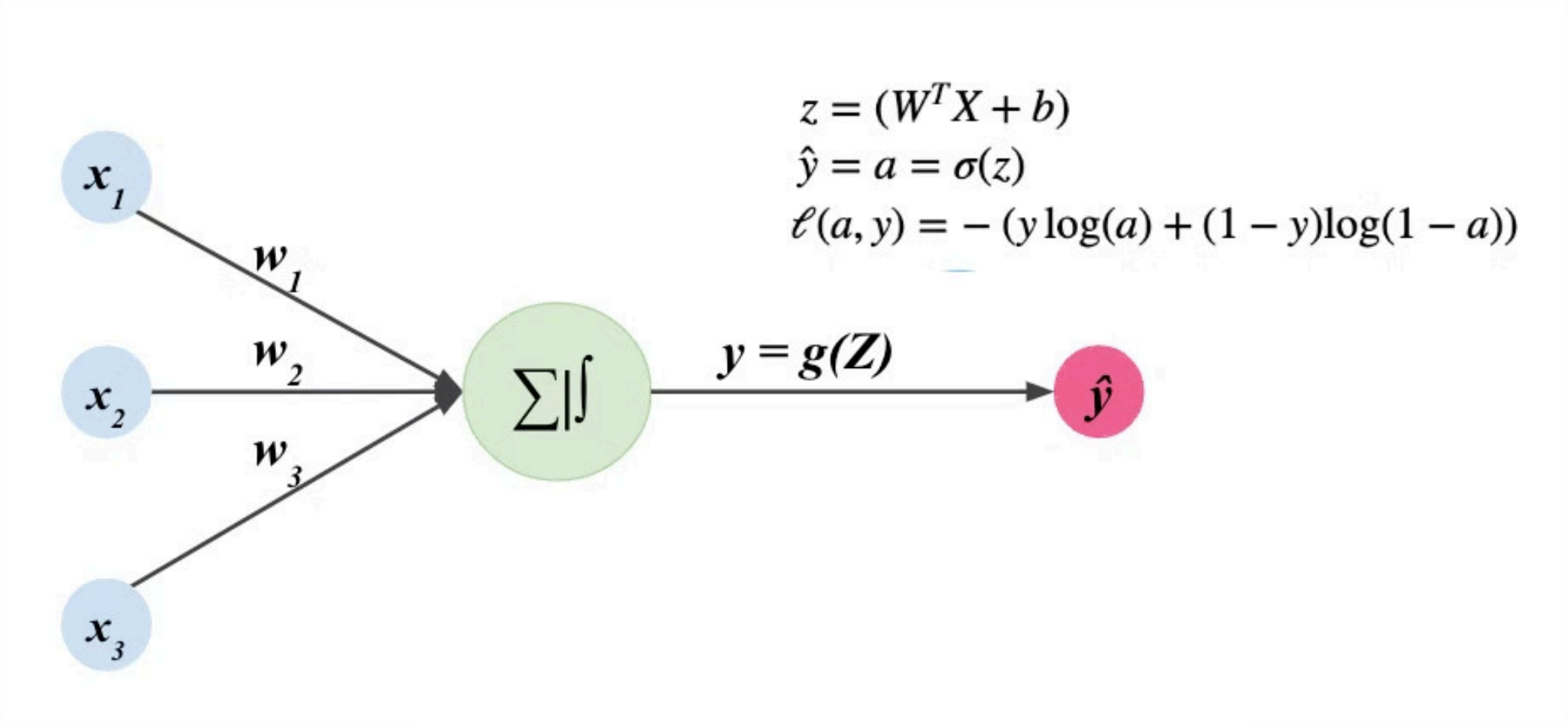


Many-to-Many | Machine Translation

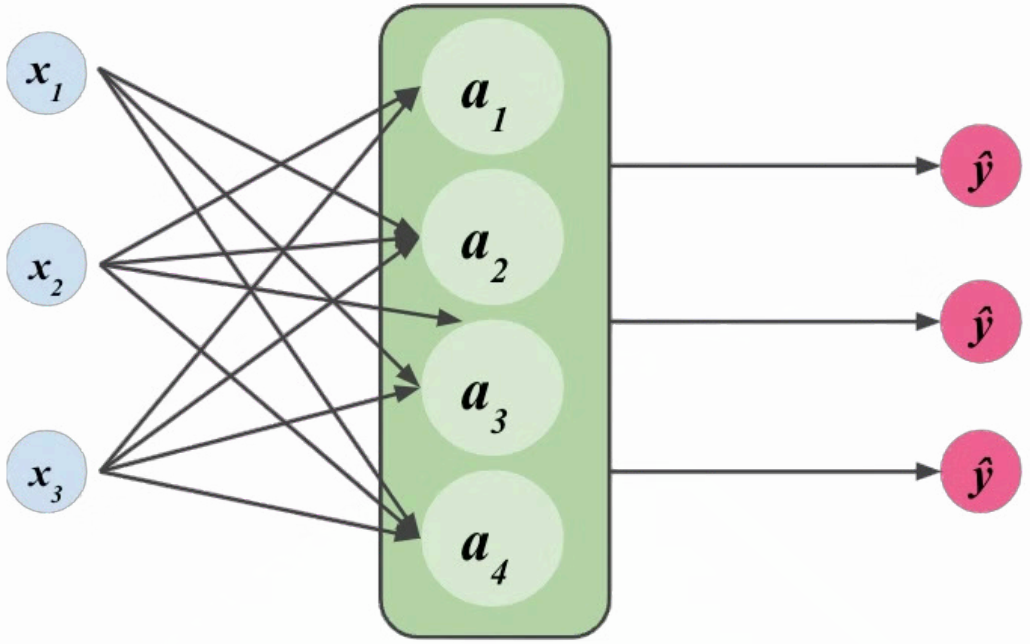


From Feed-Foward Networks (FFN) to Recurrent Neural Networks (RNN)

Perceptron


$z = (W^T X + b)$
$\hat{y} = a = \sigma(z)$
$\ell(a, y) = -(y \log(a) + (1 - y)\log(1 - a))$
x_1
x_2
x_3
w_1
w_2
w_3
$\sum|\int$
$y = g(Z)$
$\hat{y}$

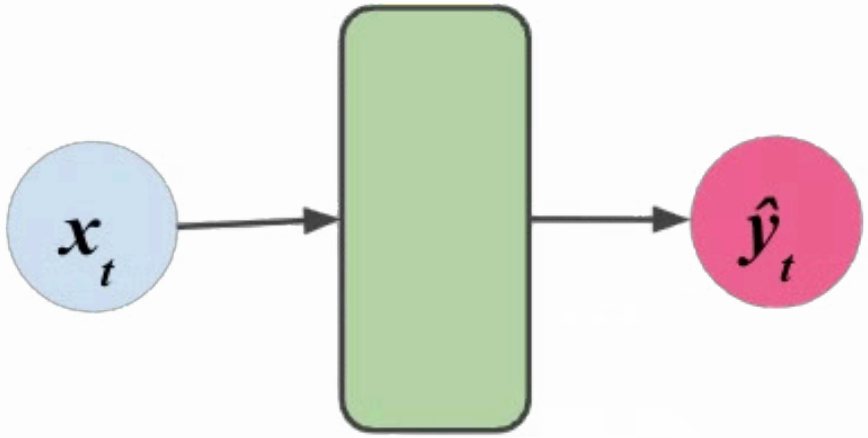
Feed-Forward Neural Network



$$x(t) \in \mathbb{R}^m$$

$$\hat{y}(t) \in \mathbb{R}^n$$

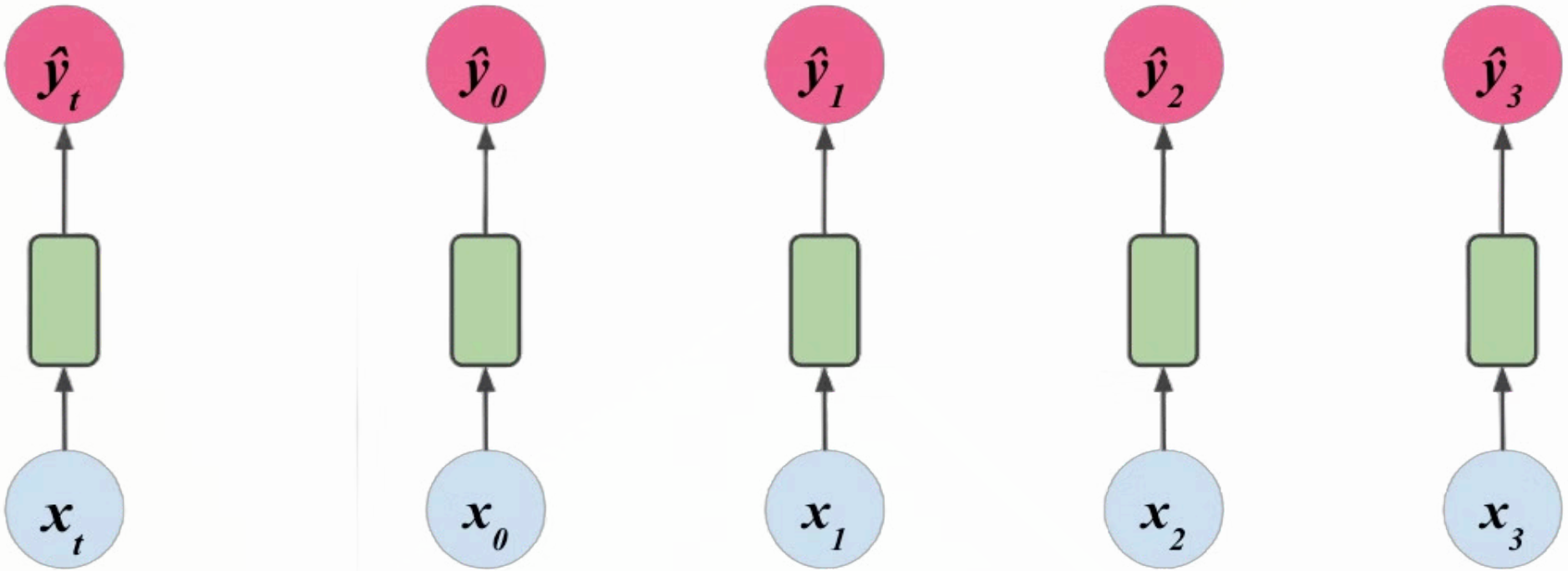
Handling Temporal Data



$$x(t) \in \mathbb{R}^m$$

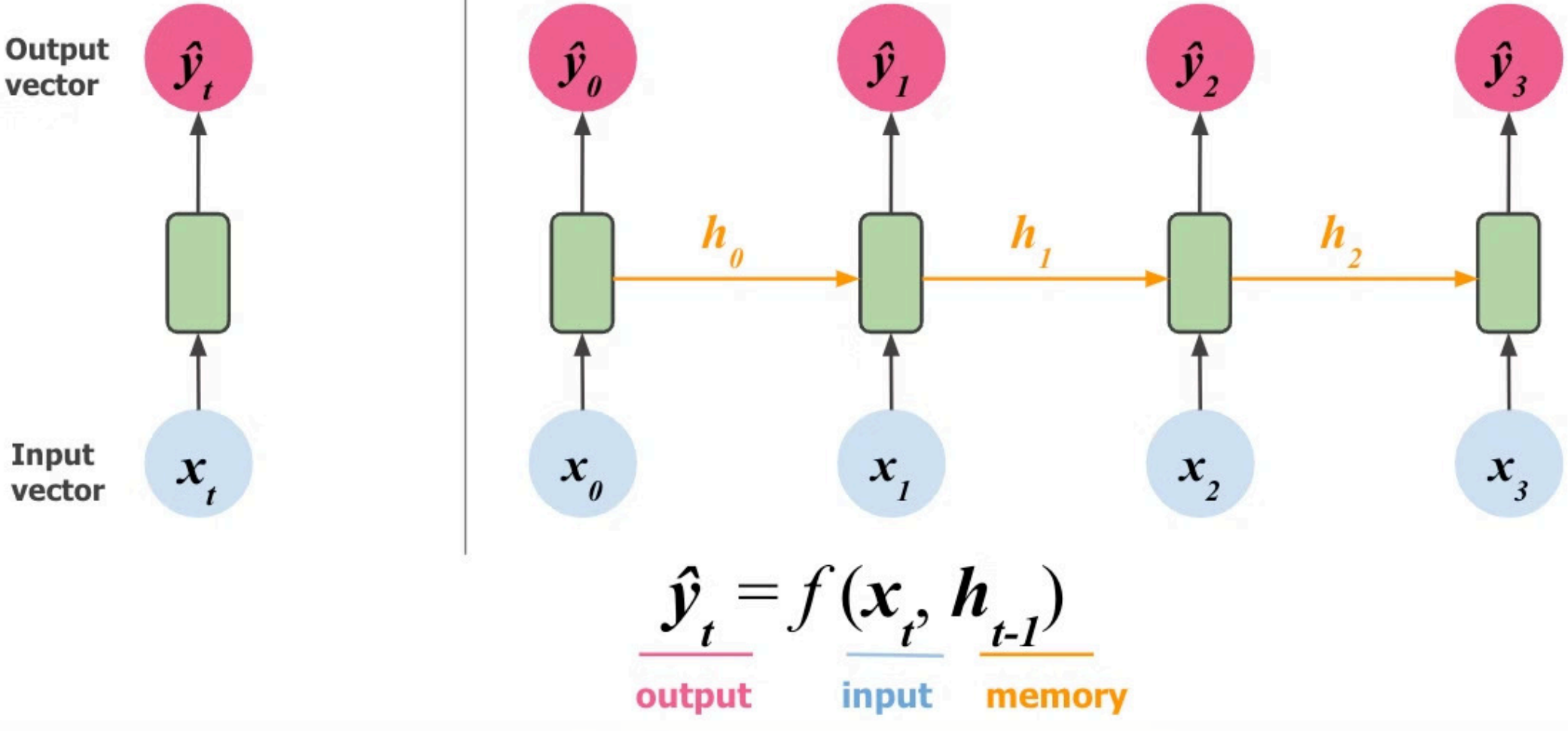
$$\hat{y}(t) \in \mathbb{R}^n$$

Dealing with Temporal Data | Sequence

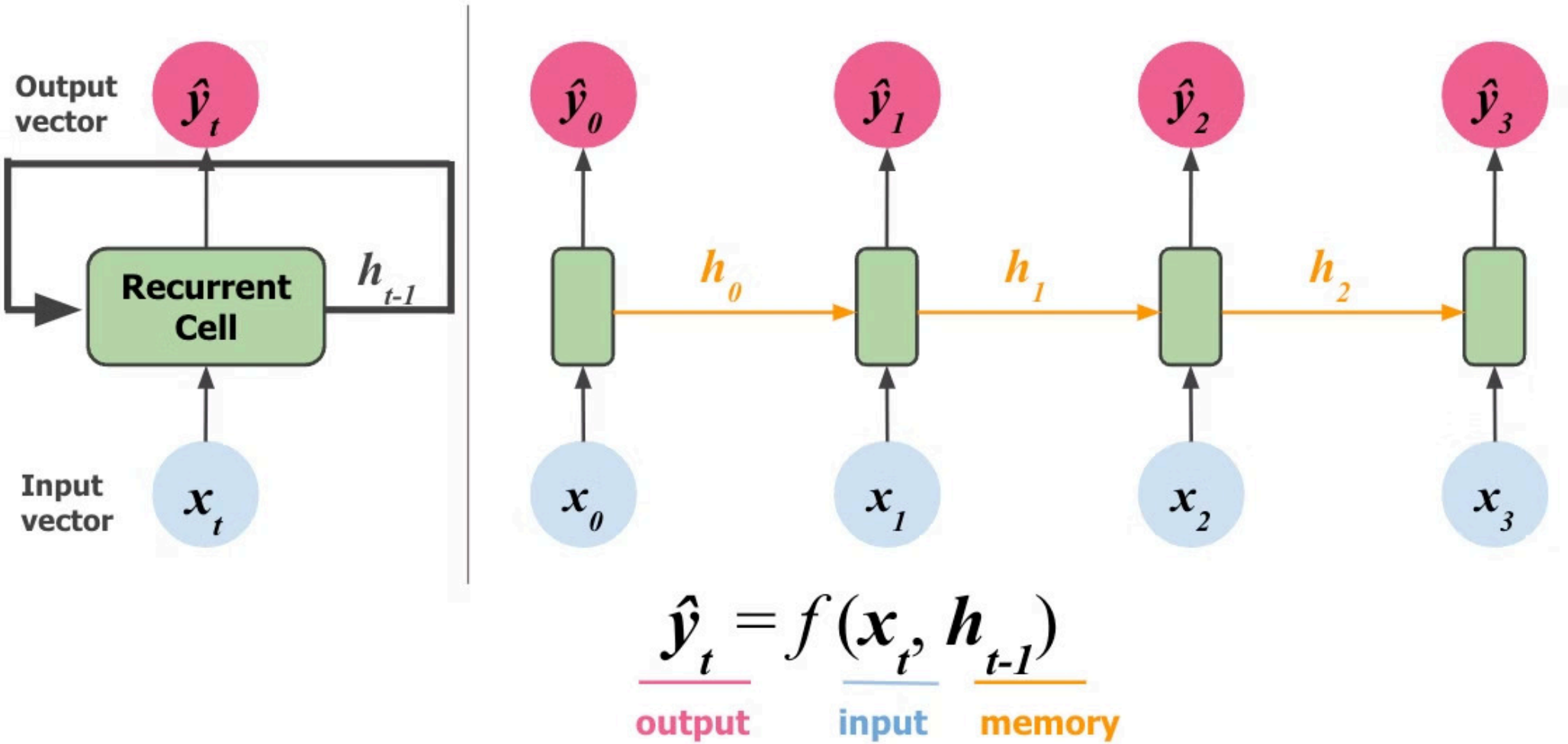


$$\hat{y}_t = f(x_t)$$

Recurrent Cells | Neurons



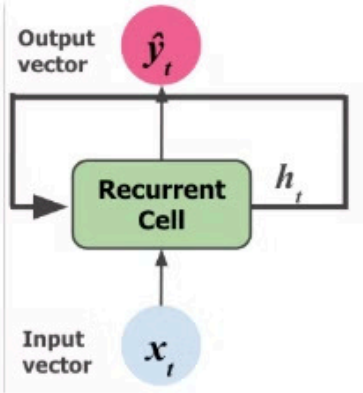
Recurrent Cells | Neurons



Next Word Prediction Intuition

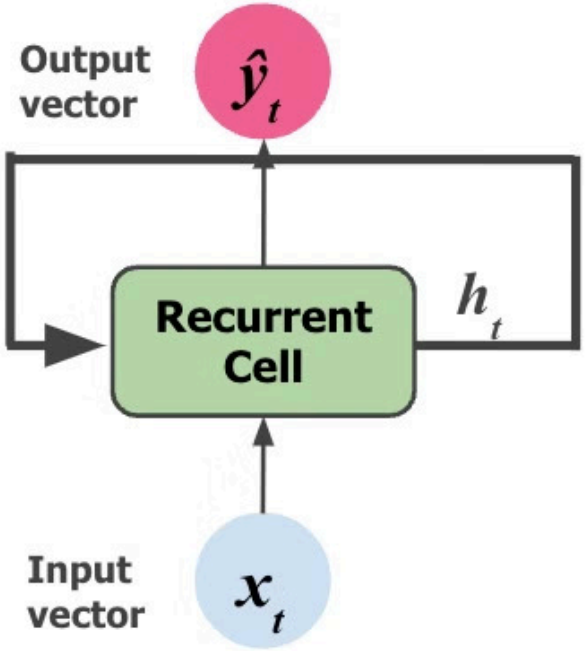
rnn.py

```python
# Weights and biases are initialized here.
weights_input = initialize_weights()
weights_hidden = initialize_weights()
weights_output = initialize_weights()

bias_hidden = initialize_bias()
bias_output = initialize_bias()

# This function is an implementation of a simple RNN.
# It takes a word and a hidden state as input and returns a prediction and the updated hidden state.

def RNN(word, hidden_state):
    # The input word is transformed into a suitable format such as one-hot encoding or word embeddings.

    # The hidden state is updated based on the current word and the previous hidden state.
    # An activation function (e.g., tanh or relu) is typically used here in an actual implementation.
    hidden_state = activation_function(weights_input * word + weights_hidden * hidden_state + bias_hidden)

    # The output prediction is computed based on the updated hidden state.
    # An activation function is typically used here in an actual implementation.
    prediction = activation_function(weights_output * hidden_state + bias_output)

    # The function returns the output prediction and the updated hidden state.
    return prediction, hidden_state

```



RNN Inference Pseudocode

```python
# Assume the weights and biases have been trained and are available.
weights_input = trained_weights_input
weights_hidden = trained_weights_hidden
weights_output = trained_weights_output

bias_hidden = trained_bias_hidden
bias_output = trained_bias_output

# Initialize the hidden state to some default value.
hidden_state = initialize_hidden_state()

function RNN_Inference(sequence):
    """
    Use a trained RNN to make predictions from a sequence of inputs.

    Parameters:
    sequence: sequence of inputs

    Returns:
    predictions: sequence of outputs
    """

    predictions = []

    # For each word in the sequence
    for word in sequence:
        # Transform the word into a suitable input format
        word = transform_input(word)

        # Compute the new hidden state and the output prediction
        hidden_state = activation_function(weights_input * word + weights_hidden * hidden_state + bias_hidden)
        prediction = activation_function(weights_output * hidden_state + bias_output)

        # Append the prediction to the list of predictions
        predictions.append(prediction)

    # Return the sequence of predictions
    return predictions
```



Sequence Modeling Design Criteria

Variable Length Sequences

Models must efficiently process sequences of different lengths, from very short to very long inputs

Long-Term Dependencies

Ability to recognize and maintain relationships between elements that are far apart in the sequence

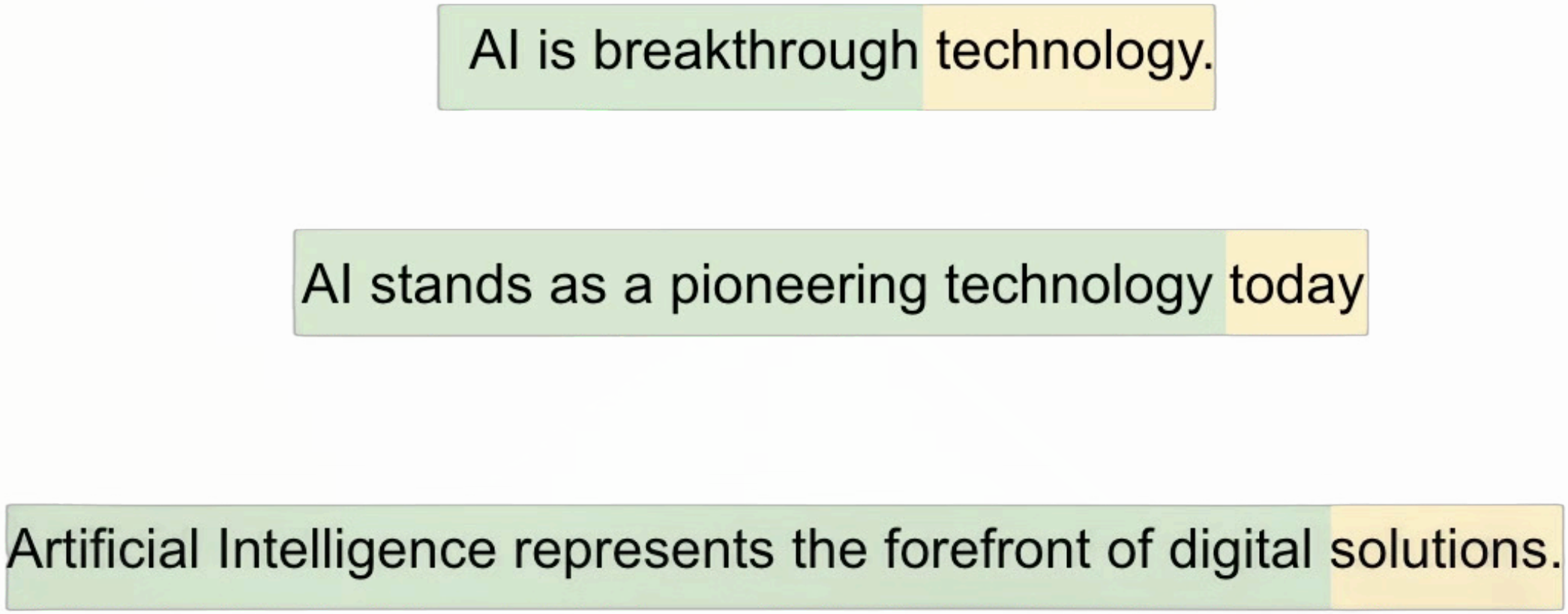
Direction of Information Flow

Flexibility to process information in one direction or both directions depending on the task

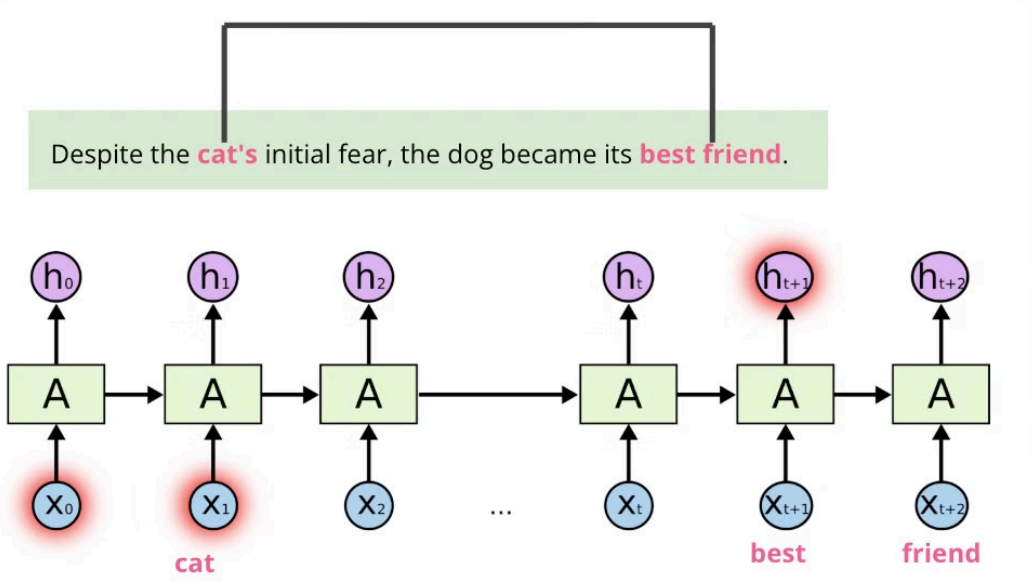
Order of Information

Preserving the sequential order of inputs for proper context and meaning

Variable Length Sequences

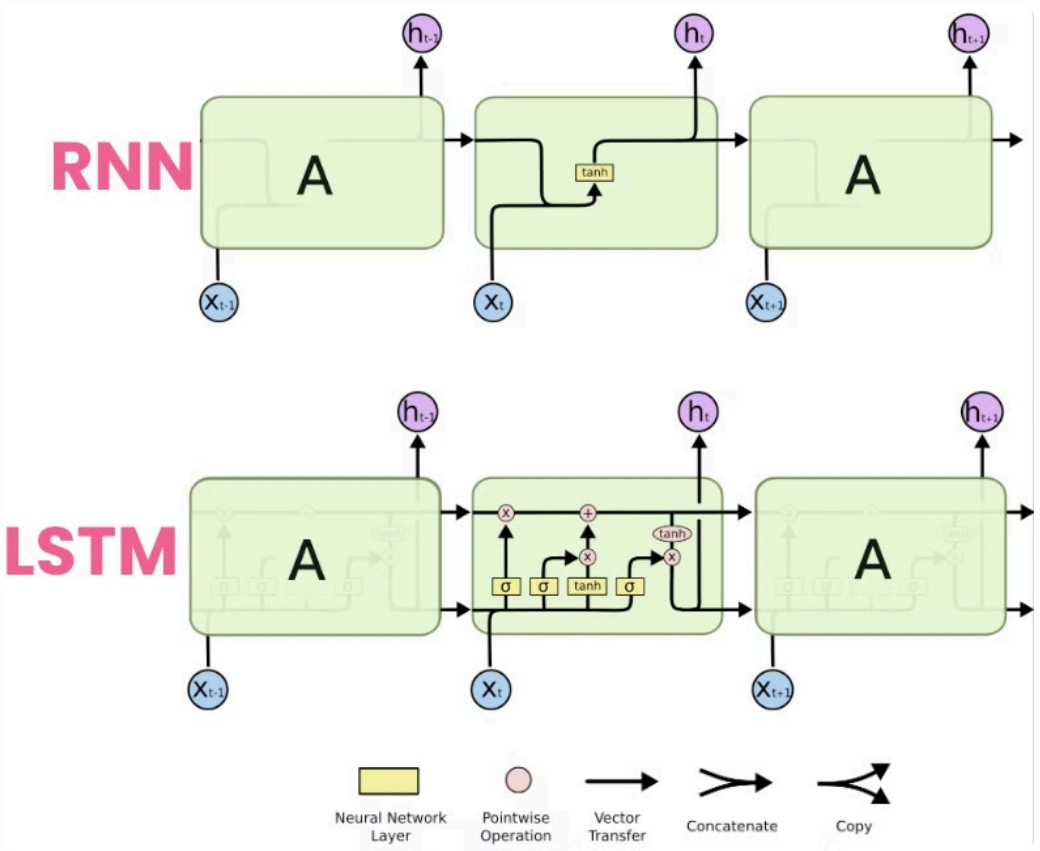


Long-Term Dependencies



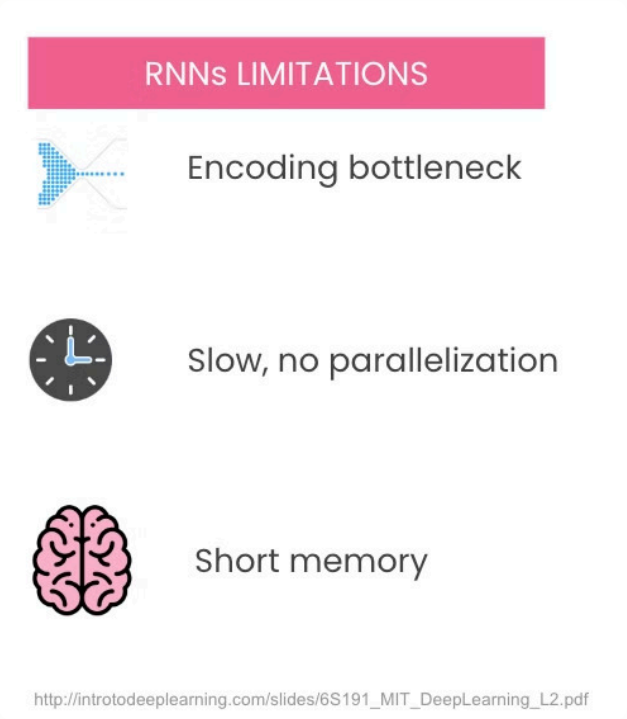
- **Core Concept:** RNNs process sequential data by passing hidden states through time steps
- Example: Early words influence later predictions in a sentence
- **Challenge:** RNNs struggle to maintain context between distant inputs and outputs
- **Key Problem:** Vanishing gradients during backpropagation weaken long-distance learning
- **Solution:** LSTMs use memory cells and gates to preserve important information across long sequences

Long-Term Dependencies



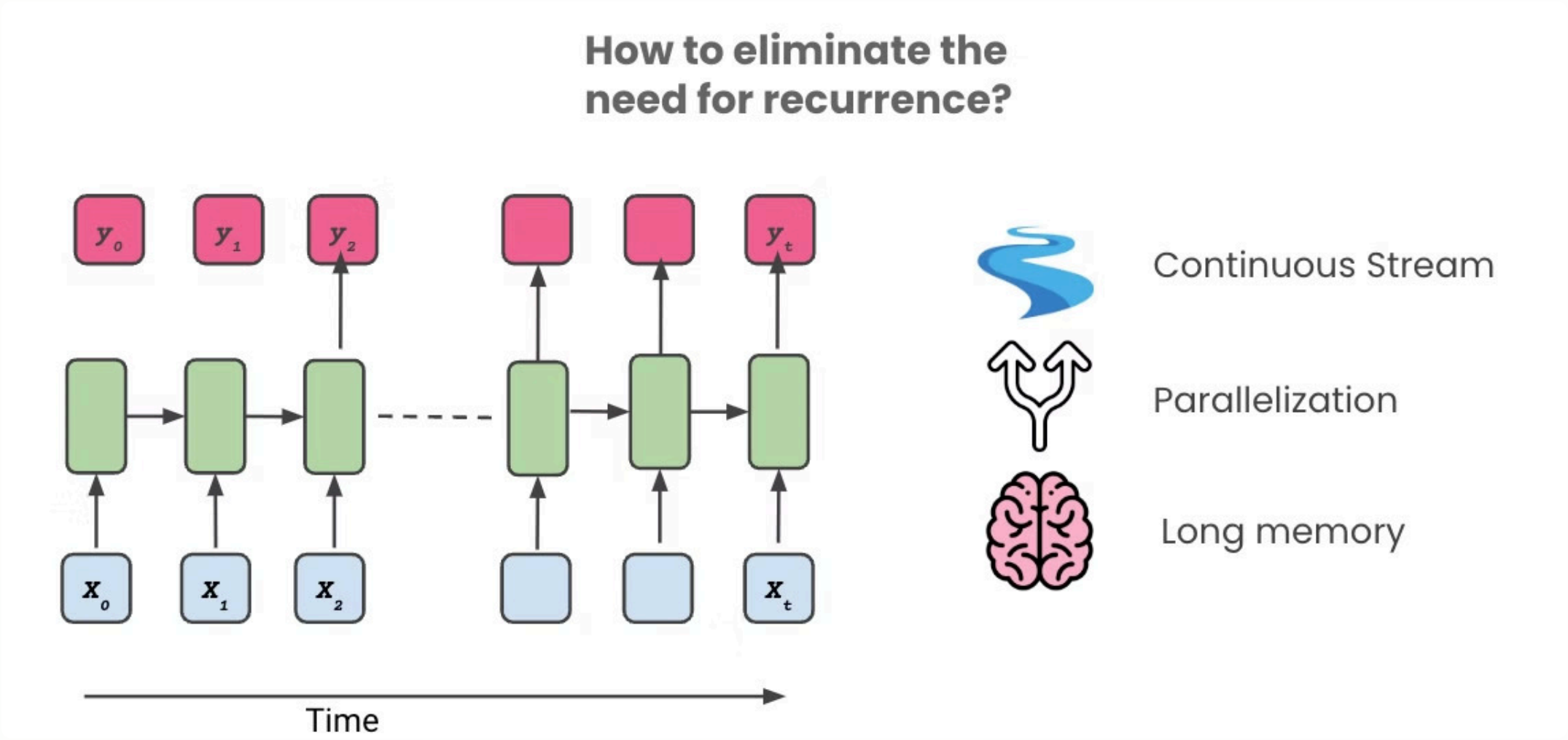
- **RNN (Recurrent Neural Network)**
  - Basic sequential processing
  - Works with short sequences only
- **LSTM (Long Short-Term Memory)**
  - Enhanced memory architecture
  - Manages extended sequences
  - Prevents gradient loss via controlled memory
- **Key Operations**
  - Selective memory gates
  - Controlled state management
  - Input-output processing
- **Overall Comparison**
  - LSTM enhances RNN capabilities
  - Higher performance at computational cost

Limitations of RNNs/LSTM

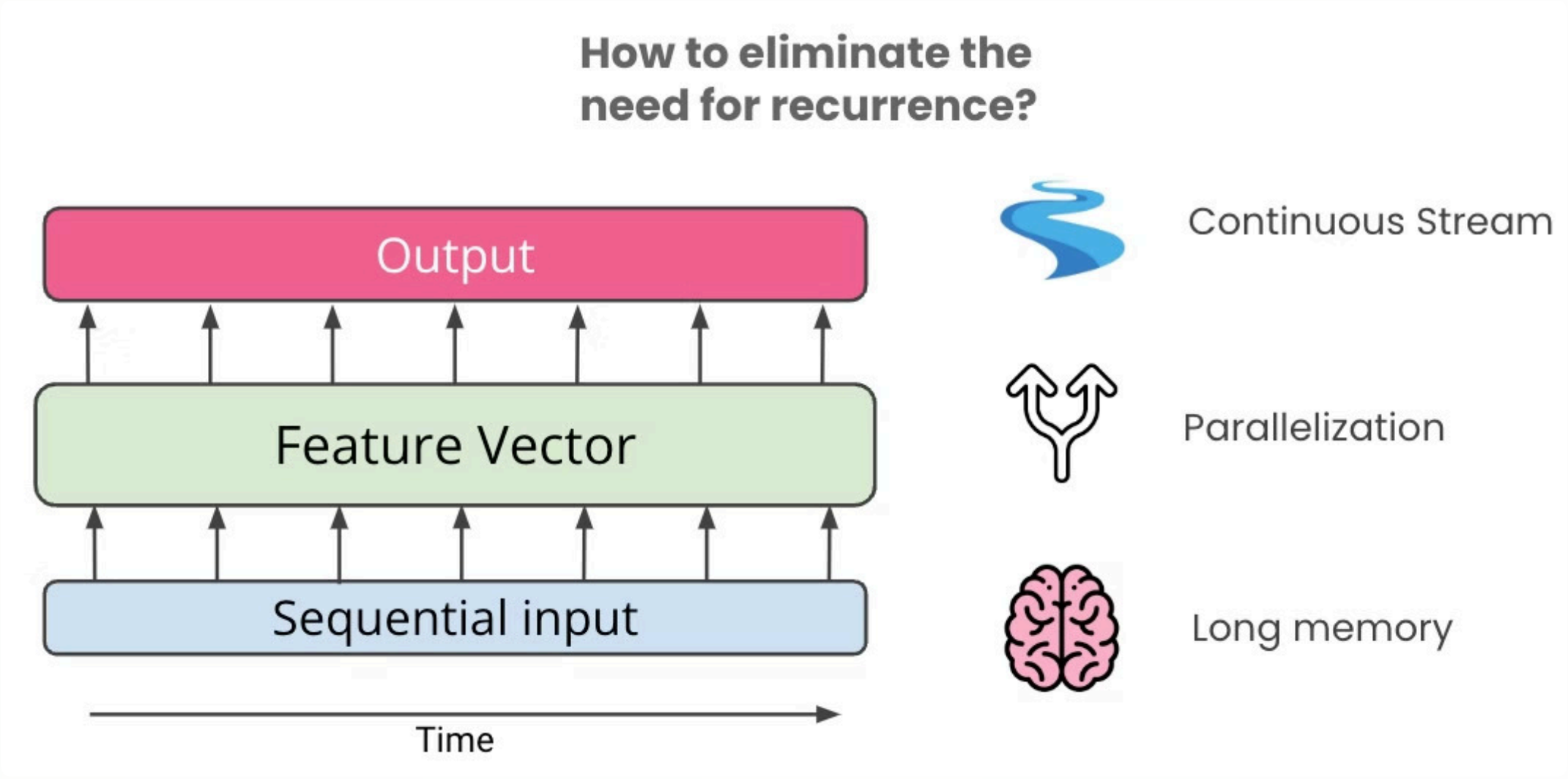


1. **Encoding Bottleneck:**
   - Compressing all sequence information into a fixed-size vector leads to loss of context, especially in long sequences.
2. **Slow and No Parallelization:**
   - Sequential nature of RNNs prevents parallel processing, making training and inference time-consuming.
3. **Short Memory:**
   - Struggles to capture long-term dependencies due to vanishing gradient problems, leading to poor performance on extended sequences.

Removing the Bottlenecks



Desired Capabilities



Attention is All You Need!

Xiv:1706.03762v5 [cs.CL] 6 Dec 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

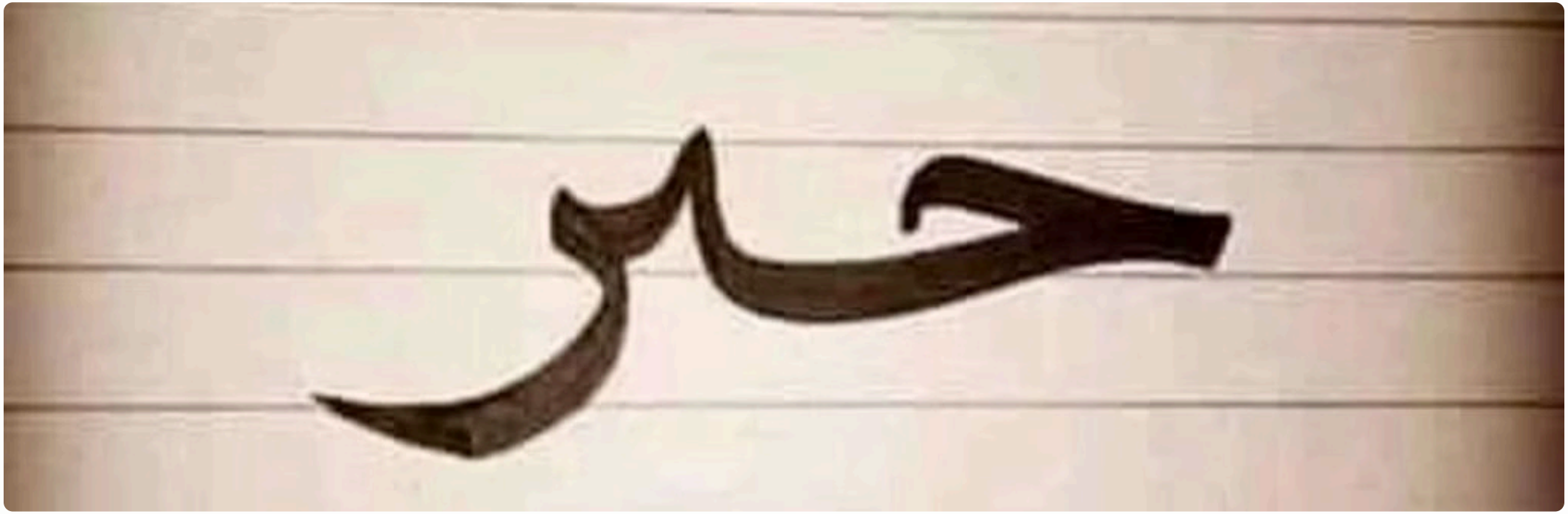
- **Significance:**
  - Marked a paradigm shift in machine learning.
  - Empowered advancements in NLP tasks like **translation, summarization, and text generation**.
  - Enabled scaling of models to unprecedented levels.

What is Attention?

Describe what you see in this picture?

What is you pay more **Attention**?

How to read this?



The Context Matters

The Context Matters

The Context Matters

قرأت
حـــــر
رائعا في الجريدة

The Context Matters

قرأت
خـــبـــرا
رائعا في الجريدة

The Context Matters

اشترى الخطاط
حـــــر
ليكتب مخطوطات

The Context Matters

اشترى الخطاط
حبرا
ليكتب مخطوطات

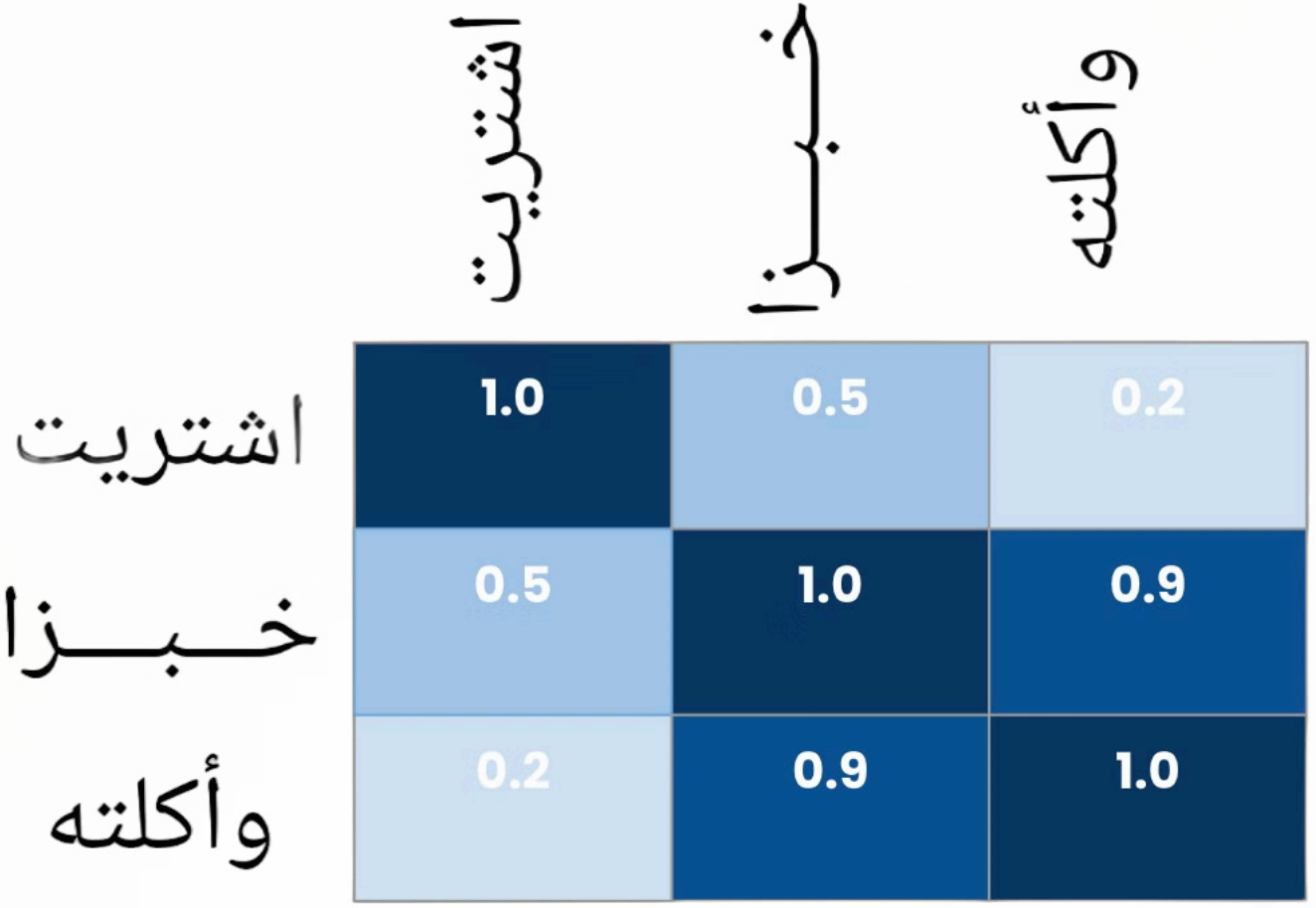
The Context Matters

لقد قام ب
جـــبـــر
خاطر هذا المسكين

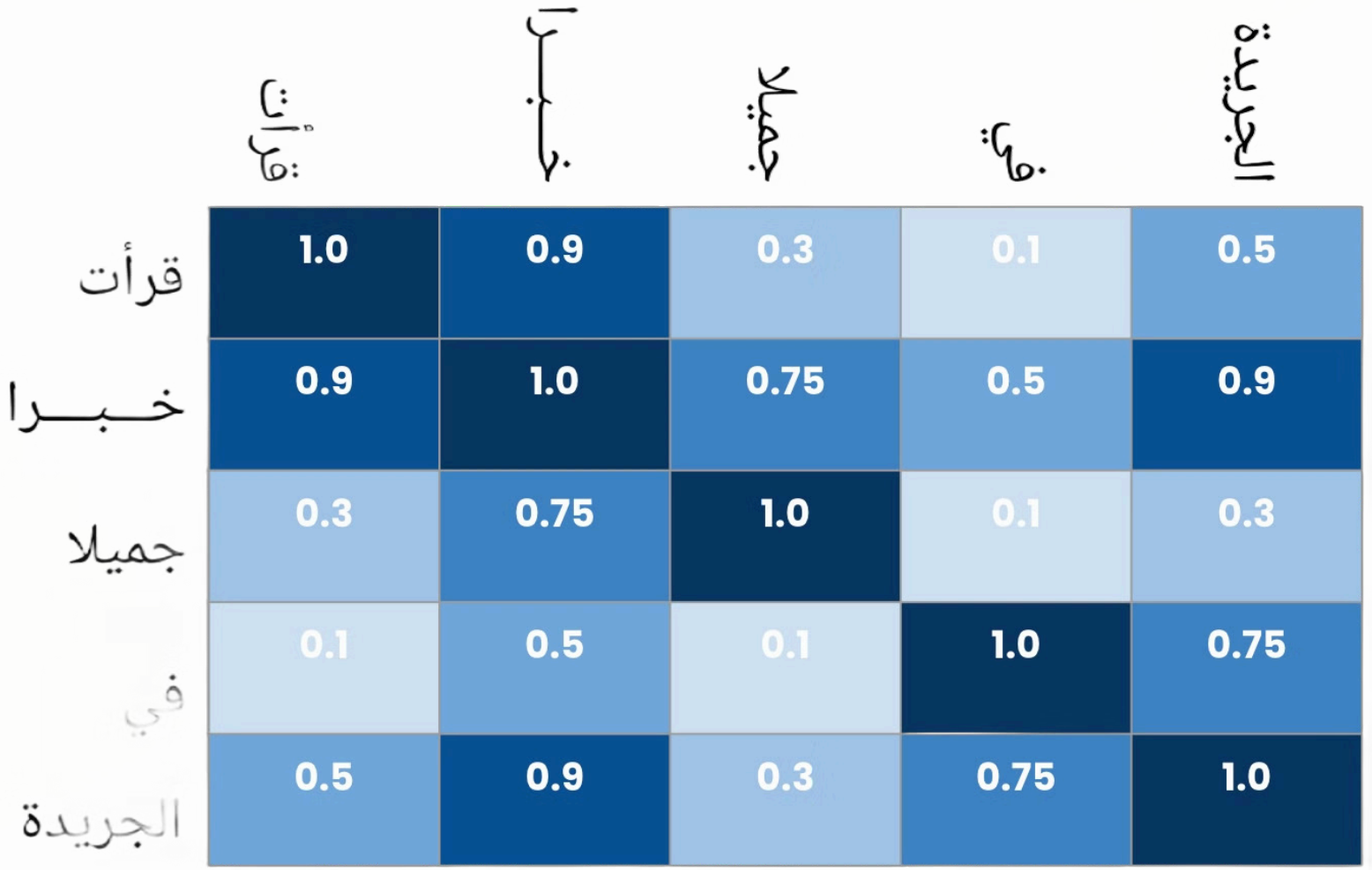
The Context Matters

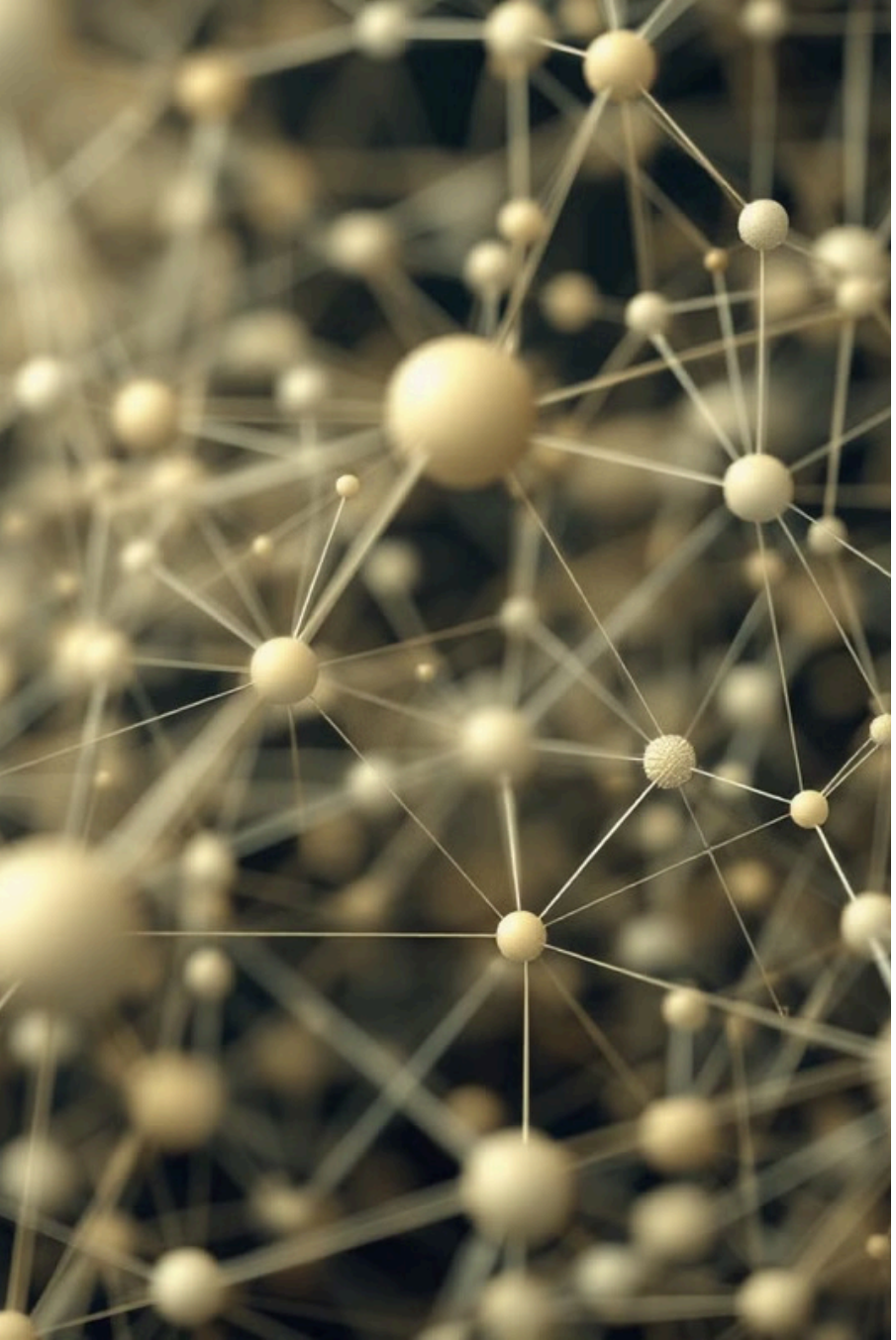
من كان يؤمن بالله واليوم
الآخر فليقل
خـــيـــرا
أو ليصمت

Capture Relation Between Words



Capture Relation Between Words





Thank You!

Thank you for your time.